# Sorting, Selection, and Randomized Analysis

Sorting determines the order of every record; selection asks for one rank. They can share partitioning and heaps without sharing the same time bound.

Read in this order: **comparison-model limits → heaps and sorting → probability tools → partitioning and quicksort → counting, radix, and bucket sorting → order statistics**. Further examples use collisions, loads, waiting times, and online selection to practice probabilistic modeling.

Prerequisites: specifications, invariants, and sums from Chapter 1; divide-and-conquer and recurrences from Chapter 2. Probability tools are developed before their use in expected-time analysis.

Reference textbook: Cormen et al., *Introduction to Algorithms*, 3rd Edition, Chapters 5–9; supplementary reference: MIT 6.046J (2005), Lectures 4–6.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# Specify the records and the sorting guarantee

A record contains a **key** and fields that move with it. Identity labels $a, b, c$ distinguish equal-key records; comparisons inspect only the key unless stated otherwise.

A correct sort produces nondecreasing keys and preserves the input multiset of records. **Stability** additionally preserves the original order of equal-key records.

For input $[2_a, 1_b, 2_c]$, output $[1_b, 2_a, 2_c]$ is stable. Output $[1_b, 2_c, 2_a]$ is sorted but unstable. Output $[1_b, 2_a, 2_a]$ loses $2_c$ and violates the sorting contract itself.

Use **1-based closed intervals** $A[p..r]$, with $p > r$ denoting an empty interval. Key-indexed counting tables may start at $0$. Selection rank $k$ is always 1-based and local to the current subarray.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Count comparisons, time, and space separately

Unless stated otherwise, comparing or moving a fixed-size record and accessing one index take constant time. Long strings, large integers, and external objects require explicit comparison, digit-access, and movement costs.

| Question | Conditions that must be fixed |
| --- | --- |
| Worst-case time $\max_A T(A)$ | Length and legal input set |
| Average-case time $\mathbb{E}_A[T(A)]$ | Joint input distribution |
| Randomized time $\mathbb{E}_R[T(A,R)]$ | Fixed input $A$; average over internal randomness $R$ |
| Auxiliary space | Whether output, buffers, and live call frames are counted |

An in-place partition does not remove quicksort's recursion stack. A small expected time does not exclude an expensive random execution.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A comparison-tree leaf must identify an ordering

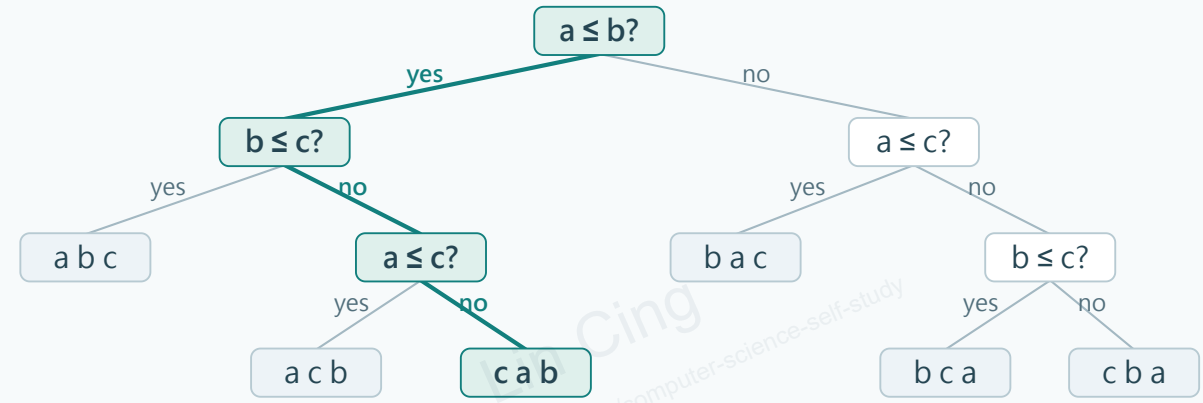
First restrict attention to $n$ distinct keys. A comparison sort learns order only through comparisons between keys, without using key values as array indices. Fix $n$ and represent all executions by a binary tree.

Each internal node identifies two original records. Its branches record $\leq$ or $>$. A root-to-leaf path records the observations, and its leaf must determine the complete identity order.

The $n!$ relative orders require distinct reachable leaves. If two orders reached the same leaf, the algorithm would observe the same outcomes and output the same identity permutation; at least one would be incorrect. Thus at least $n!$ leaves are reachable.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Follow the comparisons to the identity order




Leaf $c, a, b$ means $c < a < b$. For $a = 4, b = 7, c = 1$, follow the comparison outcomes to that leaf to check one complete execution.



## A worst-case lower bound requires some hard input

Here height $h$ counts edges on a longest root-to-leaf path; if the root is level 1, the number of levels is $h + 1$. A binary tree of height $h$ has at most $2^h$ leaves, so

$$n! \le 2^h, \qquad h \ge \lceil \lg(n!) \rceil.$$

For $n \ge 2$, at least $\lfloor n/2 \rfloor$ final factors of $n!$ are at least $n/2$, while every factor is at most $n$. Hence

$$\lfloor n/2 \rfloor \lg(n/2) \le \lg(n!) \le n \lg n.$$

These give $\lg(n!) = \Theta(n \lg n)$ for sufficiently large $n$; finitely many smaller sizes are covered separately.

Thus **every correct comparison sort has a worst-case input requiring $\Omega(n \lg n)$ comparisons**. It need not spend that much on every input: insertion sort uses $n - 1$ key comparisons on sorted data.

Allowing duplicates still includes the hard distinct-key subset. Counting and radix sorts inspect key representations and leave this model, introducing range and bit-length costs instead.

## The deepest leaf alone does not bound the average

Let the $N = n!$ relative orders be uniform, and select one representative leaf for each, at depths $d_1, \ldots, d_N$. Their root paths are prefix-free. Associate a depth-$d_i$ leaf with its binary interval of length $2^{-d_i}$ in $[0, 1)$. The intervals are disjoint, so

$$\sum_{i=1}^{N} 2^{-d_i} \leq 1.$$

The geometric mean is at most the arithmetic mean:

$$2^{-\frac{1}{N}\sum_i d_i} \leq \frac{1}{N} \sum_i 2^{-d_i} \leq \frac{1}{N}.$$

Taking logarithms yields $\frac{1}{N}\sum_i d_i \geq \lg N = \Omega(n \lg n)$. This bounds average depth under uniform inputs; the deepest path alone would not suffice.

For a randomized sort correct for every random choice $r$, this bound holds for each fixed $r$. Averaging over $r$ gives $\max_A E_r[C(A, r)] \geq \lg(n!)$, since a maximum is at least the uniform-input average.



## Three keys: distinguish the two averaging domains

Example: choose fairly between the earlier three-key tree L and its identity-renamed version R (input order $a, c, b$).

| Identity order | Tree L | Tree R | Fair-choice mean |
|---|---|---|---|
| $a, b, c$ | 2 | 3 | $5/2$ |
| $a, c, b$ | 3 | 2 | $5/2$ |
| $b, a, c$ | 2 | 3 | $5/2$ |
| $b, c, a$ | 3 | 3 | 3 |
| $c, a, b$ | 3 | 2 | $5/2$ |
| $c, b, a$ | 3 | 3 | 3 |

Uniform-input column means equal $16/6 = 8/3 \geq \lg 6$; the maximum fixed-input row mean is three. The averaging domains differ.

## A heap has both a shape rule and a key-order rule

A binary heap is a **complete binary tree**: all levels before the last are full, and the last fills from left to right. In a 1-based array,

$$parent(i) = \lfloor i/2 \rfloor, \quad left(i) = 2i, \quad right(i) = 2i + 1.$$

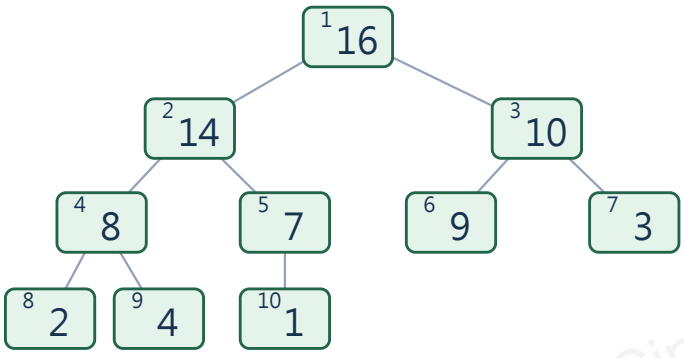
Only positions $1 \le i \le s$ belong to the current heap, where $s$ is its heap-size. Array capacity, array length, and $s$ are separate quantities.

A max-heap requires $A[parent(i)] \ge A[i]$ for every nonroot position $i$. Here $A[i]$ abbreviates its key; swaps move entire records. Following ancestors proves the root is a maximum. Levels and different subtrees need not be sorted.

Height counts **edges** on a longest downward path: a singleton has height 0, and a nonempty $n$-node heap has height $\lfloor \lg n \rfloor$. Counting levels instead gives $\lfloor \lg n \rfloor + 1 = \lceil \lg(n+1) \rceil$. Handle an empty heap separately, without evaluating $\lg 0$.



## One heap, two representations




Node: small index, large key

Fixed indices; moving keys

| 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|
| 16 | 14 | 10 | 8 | 7 |
| 6 | 7 | 8 | 9 | 10 |
| 9 | 3 | 2 | 4 | 1 |

Every edge: parent key ≥ child key
One level need not be sorted

Node labels are array positions. Swaps move keys between positions; a parent index stays fixed.

## State the precondition for sifting down

Precondition: $1 \le i \le s$, both child subtrees are already max-heaps, and only edges from $i$ to its children may violate order. Missing children are not read; `and` uses short-circuit evaluation.

```
MAX-HEAPIFY(A,i,s):
    while true:
        largest = i
        if 2*i <= s and A[2*i] > A[largest]:
            largest = 2*i
        if 2*i+1 <= s and A[2*i+1] > A[largest]:
            largest = 2*i+1
        if largest == i: return
        swap(A[i],A[largest])
        i = largest
```

Each iteration finds a maximum among the current node and its existing children before deciding whether to swap. Comparing the two children alone cannot determine whether their parent needs repair.



## Why sifting down is correct and terminates

If the current node is a maximum, its child edges satisfy heap order. The two subtrees already satisfy it, so the entire subtree is repaired.

Otherwise promote the larger child root. It dominates its descendants, the other child root, and the old parent key. Both new edges from the parent are valid. Only the old parent's new position may violate order; the other subtree is unchanged.

Induction on remaining height proves correctness. Each swap changes $i$ to $2i$ or $2i+1$, moving down one level. At height $h$, there are at most $h$ descents, giving $O(h+1)$ time and $O(1)$ auxiliary space for this iterative version.

For $A = [16, 4, 10, 8, 7, 9, 3]$, $i = 2$, $s = 7$, choose child key $8$ and swap positions $2, 4$, producing $[16, 8, 10, 4, 7, 9, 3]$. Child indices $8, 9$ of position $4$ are out of range, so stop. Key $3$ at index $7$ belongs to the other subtree and is not compared in this descent.

## Build bottom-up so the child subtrees are ready

```
BUILD-MAX-HEAP(A):
    s = length(A)
    for i = floor(s/2) downto 1:
        MAX-HEAPIFY(A,i,s)
```

Invariant: at each iteration's start, every node whose index exceeds $i$ roots a valid max-heap.

Initially $i = \lfloor n/2 \rfloor$, and all larger indices are leaves. Child indices exceed $i$, so MAX-HEAPIFY's precondition holds. It repairs $i$ while preserving all descendant heaps. Decrementing $i$ restores the invariant for the next iteration.

At $i = 0$, root $1$ is valid and the entire heap is complete. For $n = 0, 1$, the loop is skipped and the empty or singleton heap is already valid.

Index $\lfloor n/2 \rfloor$ is the last possible internal node, **not the first leaf**. For $n = 10$, position $5$ still has left child $10$ and must be processed.



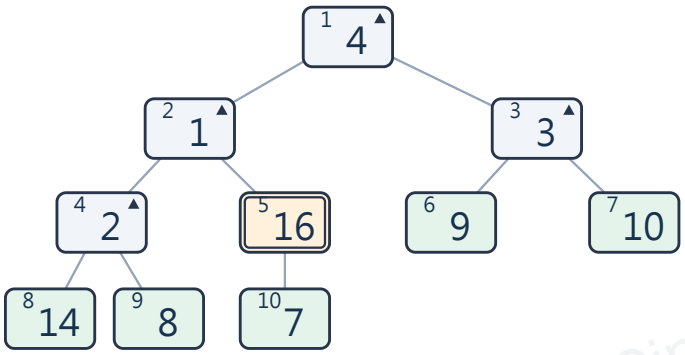
## Build backward from position five: complete states

Start with $A = [4, 1, 3, 2, 16, 9, 10, 14, 8, 7]$, active size $s = 10$. All indices are **one-based**. Each root is processed only after both child subtrees have been repaired.

| Root $i$ | Child choice and exchanged indices | Repaired $A[1..10]$ |
|---|---|---|
| $5$ | Only child key $7$; $16 \geq 7$, no swap | $[4, 1, 3, 2, 16, 9, 10, 14, 8, 7]$ |
| $4$ | Choose $14$ over $8$; swap $4 \leftrightarrow 8$ | $[4, 1, 3, 14, 16, 9, 10, 2, 8, 7]$ |
| $3$ | Choose $10$ over $9$; swap $3 \leftrightarrow 7$ | $[4, 1, 10, 14, 16, 9, 3, 2, 8, 7]$ |
| $2$ | Choose $16$, then $7$; swap $2 \leftrightarrow 5$, $5 \leftrightarrow 10$ | $[4, 16, 10, 14, 7, 9, 3, 2, 8, 1]$ |
| $1$ | Choose $16, 14, 8$; swap $1 \leftrightarrow 2$, $2 \leftrightarrow 4$, $4 \leftrightarrow 9$ | $[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$ |

The following tree-and-array figures use these same records. Orange marks the root being processed; green marks certified heap roots. Position relationships stay fixed while swaps move keys and their associated records.

## Position five has only a left child




Node: small index, large key

Fixed indices; moving keys

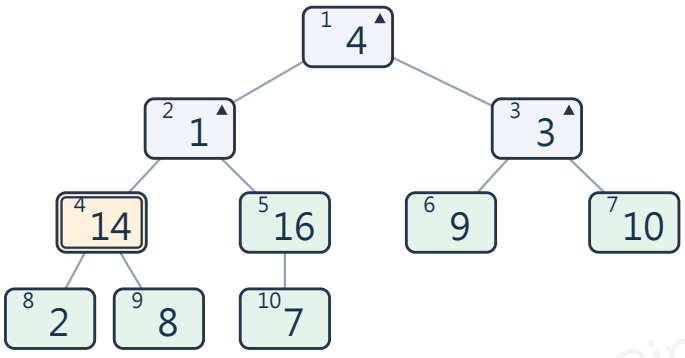
| 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|
| 4 | 1 | 3 | 2 | 16 |
| 6 | 7 | 8 | 9 | 10 |
| 9 | 10 | 14 | 8 | 7 |

Double frame: root just repaired
No mark: legal; triangle: pending

Key $16$ at position $5$ already dominates key $7$ at left-child position $10$, so no swap occurs. Right-child index $11 > s$ must not be read. Positions $6..10$ are leaves; the repaired subtree at five extends the bottom-up invariant.



## Position four promotes the larger child, fourteen




Node: small index, large key

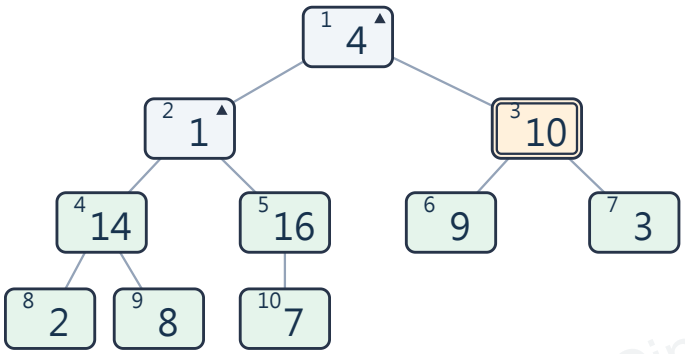
Fixed indices; moving keys

| 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|
| 4 | 1 | 3 | 14 | 16 |
| 6 | 7 | 8 | 9 | 10 |
| 9 | 10 | 2 | 8 | 7 |

Double frame: root just repaired
No mark: legal; triangle: pending

Position four contains key two; children at eight and nine contain fourteen and eight. Choose fourteen, compare it with two, then swap positions four and eight. The old key reaches leaf eight and stops; key eight at position nine is unchanged.

## Position three chooses ten over nine




Node: small index, large key

Fixed indices; moving keys

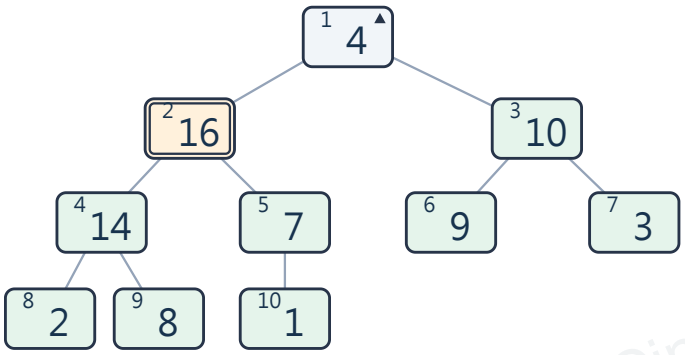
| 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|
| 4 | 1 | 10 | 14 | 16 |
| 6 | 7 | 8 | 9 | 10 |
| 9 | 3 | 2 | 8 | 7 |

Double frame: root just repaired
No mark: legal; triangle: pending

Position three has key three; children six and seven contain nine and ten. Choose ten and swap positions three and seven. The old key stops at leaf seven; key nine at six is unchanged. Previously valid child heaps now combine into a valid heap rooted at three.



## Position two sends the old key through five and ten




Node: small index, large key

Fixed indices; moving keys

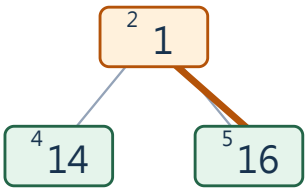
| 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|
| 4 | 16 | 10 | 14 | 7 |
| 6 | 7 | 8 | 9 | 10 |
| 9 | 3 | 2 | 8 | 1 |

Double frame: root just repaired
No mark: legal; triangle: pending

Position two has child keys fourteen and sixteen; swap with sixteen first. The old key one reaches position five, remains below its sole child seven, and swaps again with position ten. The path is $2 \to 5 \to 10$; key fourteen at position four does not move.

## Follow the same old key through local child choices

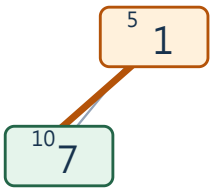
Comparison 1




16 > 1 → swap(2,5)

A[2]=16; A[5]=1

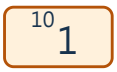
Comparison 2




7 > 1 → swap(5,10)

A[5]=7; A[10]=1

Leaf: stop




20, 21 > heap-size 10

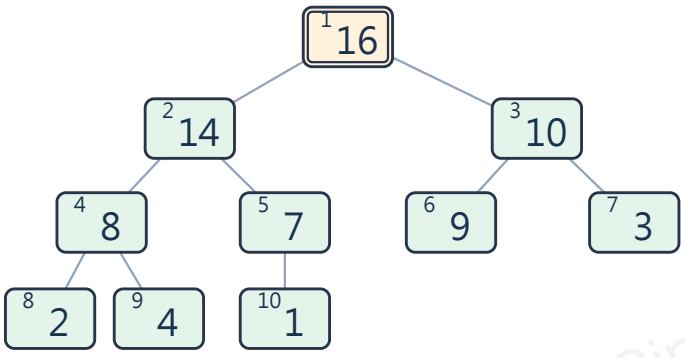
No child access

Old key 1 ends at index 10

Orange follows the old key one; the heavy edge identifies the chosen larger child. Index labels and key values are separate. First choose child five over four; at position five, only child ten exists. Unshown subtrees remain valid and need no rebuilding.



## Position one sends the old root through two, four, and nine




Node: small index, large key

Fixed indices; moving keys

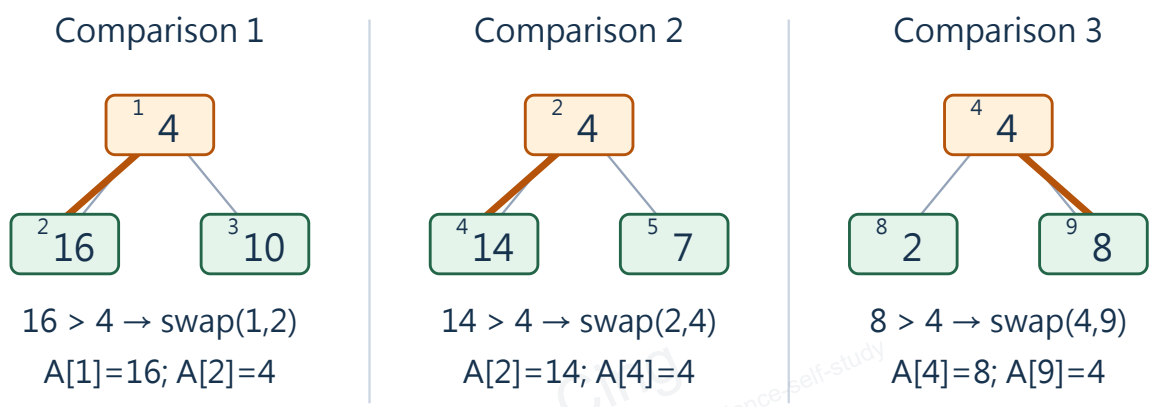
| 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|
| 16 | 14 | 10 | 8 | 7 |
| 6 | 7 | 8 | 9 | 10 |
| 9 | 3 | 2 | 4 | 1 |

Double frame: root just repaired

No mark: legal; triangle: pending

The old root key four swaps with sixteen, fourteen, then eight, stopping at leaf nine. Final parent-child checks are $16 \geq 14, 10$, $14 \geq 8, 7$, $10 \geq 9, 3$, $8 \geq 2, 4$, and $7 \geq 1$. They cover all nine tree edges, certifying the heap while preserving all ten records.

## Three downward steps: distinguish positions from keys




Each panel shows local state before a swap and the resulting keys below it. Positions follow $1 \to 2 \to 4 \to 9$, while the same old key remains four; chosen child keys are sixteen, fourteen, and eight. At position nine, child indices eighteen and nineteen exceed $s = 10$, so the repair stops.



## The complete array after every exchange

Continue after repairing position three. All array indices stay fixed at $1..10$:

| Root being repaired | Exchanged indices | Resulting $A[1..10]$ |
|---|---|---|
| 2 | $2 \leftrightarrow 5$ | $[4, 16, 10, 14, 1, 9, 3, 2, 8, 7]$ |
| 2 | $5 \leftrightarrow 10$ | $[4, 16, 10, 14, 7, 9, 3, 2, 8, 1]$ |
| 1 | $1 \leftrightarrow 2$ | $[16, 4, 10, 14, 7, 9, 3, 2, 8, 1]$ |
| 1 | $2 \leftrightarrow 4$ | $[16, 14, 10, 4, 7, 9, 3, 2, 8, 1]$ |
| 1 | $4 \leftrightarrow 9$ | $[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$ |

At each step, choose the largest valid child before comparing it with the parent. One swap changes only two cells; eight other cells and the record multiset stay unchanged. Keep intermediate states to reconstruct a downward path: the final array alone does not record its history.

## Linear-time building by summing node heights

For $n \geq 1$, position $i$ has height at least $h$ exactly when its leftmost depth-$h$ descendant exists: $i2^h \leq n$. Hence **exactly** $\lfloor n/2^h \rfloor$ nodes have height at least $h$.

Write each height as a sum of unit layers and exchange the finite sums:

$$\sum_{i=1}^{n} height(i) = \sum_{h=1}^{\lfloor \lg n \rfloor} \#\{i : height(i) \geq h\} = \sum_{h=1}^{\lfloor \lg n \rfloor} \left\lfloor \frac{n}{2^h} \right\rfloor < n.$$

One repair costs $O(height(i) + 1)$ and there are $\lfloor n/2 \rfloor$ calls, giving $O(n)$ total time. For $n \geq 2$, the loop itself performs at least $\lfloor n/2 \rfloor$ constant-work iterations, supplying $\Omega(n)$ and therefore $\Theta(n)$.

For $n = 15$, the height sum is $7 + 3 + 1 = 11$. Most nodes are not as high as the root. For an empty heap, the height sum is zero and no repair calls occur; return immediately without evaluating $\lg 0$ or asserting $0 < 0$. Building instead by repeated insertion can require $\Theta(n \lg n)$ because new keys may bubble to the root; that is a different construction algorithm.



## Heapsort shrinks the active prefix

```
HEAPSORT(A):
    BUILD-MAX-HEAP(A)
    for end = length(A) downto 2:
        swap(A[1],A[end])
        MAX-HEAPIFY(A,1,end-1)
```

At each iteration's start, $A[1..end]$ is a max-heap. Suffix $A[end + 1..n]$ is sorted increasingly and every suffix key is at least every heap key.

The swap moves the active maximum to $A[end]$, extending the sorted suffix. With heap-size $end - 1$, the old root's two subtrees remain valid, so only the new root needs repair. Swaps preserve the record multiset. At termination, the remaining singleton prefix completes the sorted array.

Building costs $O(n)$, followed by at most $n - 1$ repairs of $O(\lg(n + 1))$ time each. Total time is $O(n \lg(n + 1))$, with worst case $\Theta(n \lg n)$ from the comparison-sort lower bound. This implementation stops on equal keys and can take only $\Theta(n)$ on all-equal input; a worst-case bound need not describe every input.

Iterative repair uses $O(1)$ auxiliary space. Recursive repair requires additional stack space.

## Ten-record heapsort: nine maximum extractions

Start from $[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$. Each round first exchanges position one with the old last position $s + 1$, moving the maximum to the sorted suffix, then repairs the prefix of new size $s$. Paths below record only repair swaps, excluding the root-to-end exchange.

| New size $s$ | Repair path | Repaired heap prefix | Sorted suffix |
|---|---|---|---|
| $9$ | $1 \to 2 \to 4 \to 9$ | $[14, 8, 10, 4, 7, 9, 3, 2, 1]$ | $[16]$ |
| $8$ | $1 \to 3 \to 6$ | $[10, 8, 9, 4, 7, 1, 3, 2]$ | $[14, 16]$ |
| $7$ | $1 \to 3 \to 7$ | $[9, 8, 3, 4, 7, 1, 2]$ | $[10, 14, 16]$ |
| $6$ | $1 \to 2 \to 5$ | $[8, 7, 3, 4, 2, 1]$ | $[9, 10, 14, 16]$ |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Finish the remaining five extractions

| New size $s$ | Repair path | Repaired heap prefix | Sorted suffix |
|---|---|---|---|
| $5$ | $1 \to 2 \to 4$ | $[7, 4, 3, 1, 2]$ | $[8, 9, 10, 14, 16]$ |
| $4$ | $1 \to 2$ | $[4, 2, 3, 1]$ | $[7, 8, 9, 10, 14, 16]$ |
| $3$ | $1 \to 3$ | $[3, 2, 1]$ | $[4, 7, 8, 9, 10, 14, 16]$ |
| $2$ | $1 \to 2$ | $[2, 1]$ | $[3, 4, 7, 8, 9, 10, 14, 16]$ |
| $1$ | — | $[1]$ | $[2, 3, 4, 7, 8, 9, 10, 14, 16]$ |

In round two, the old final key one moves to the root and descends through $1 \to 3 \to 6$, swapping with ten and then nine. Every suffix is ascending and dominates every active heap key. The final prefix and suffix together form the complete ascending sequence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Priority queues: maximum, extraction, increase, and insertion

$Q.A[1..Q.s]$ is a max-heap with writable capacity `Q.capacity`. The fixed-capacity code checks empty/full conditions first. Swaps move whole records; keys determine priority.

```
MAXIMUM(Q):
    if Q.s == 0: return EMPTY
    return Q.A[1]

EXTRACT-MAX(Q):
    if Q.s == 0: return EMPTY
    result = Q.A[1]; Q.A[1] = Q.A[Q.s]; Q.s -= 1
    if Q.s > 0: MAX-HEAPIFY(Q.A,1,Q.s)
    return result
```

`MAXIMUM` costs $O(1)$. `EXTRACT-MAX` saves the maximum record, replaces the root with the last record, decreases active size, and repairs downward in worst-case $O(\lg(s+1))$. Extracting a singleton leaves size zero and does not inspect a new root.



## Increase a key: repair only toward ancestors

```
SIFT-UP(Q,i):
    while i > 1 and key(Q.A[floor(i/2)]) < key(Q.A[i]):
        swap(Q.A[i],Q.A[floor(i/2)])
        i = floor(i/2)

INCREASE-KEY(Q,i,k):
    require 1 <= i <= Q.s and k >= key(Q.A[i])
    set-key(Q.A[i],k)
    SIFT-UP(Q,i)
```

An increase can violate only ancestor edges: the new key still dominates its old children. Each exchange repairs the current parent edge and moves the candidate upward. Stop at the root or a dominating parent, in worst-case $O(\lg(s+1))$. Evaluate `and` left to right with short-circuiting, checking $i > 1$ first.

## Insert directly without a sentinel value

```
INSERT(Q,record):
    if Q.s == Q.capacity: return FULL
    Q.s += 1
    Q.A[Q.s] = record
    SIFT-UP(Q,Q.s)
```

Write the new record directly into the new final position and sift upward; no negative-infinity sentinel is needed. Nonfull fixed-capacity insertion costs worst-case $O(\lg(s+1))$. Dynamic-array reallocation can move $\Theta(s)$ records in one operation and needs a separate amortized analysis.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Trace the increase from position eight

Increase-key follows parent indices 8 → 4 → 2

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| Original heap | 20 | 15 | 18 | 7 | 12 | 9 | 17 | 2 |
| Write 16 at 8 | 20 | 15 | 18 | 7 | 12 | 9 | 17 | **16** |
| Swap 8 ↔ 4 | 20 | 15 | 18 | **16** | 12 | 9 | 17 | 7 |
| Swap 4 ↔ 2 | 20 | **16** | 18 | 15 | 12 | 9 | 17 | 7 |

Compare 16 with parents 7, 15, then 20; stop because 20 ≥ 16.

After writing sixteen at eight, compare parent keys seven, fifteen, and twenty. Exchanges $8 \leftrightarrow 4$ and $4 \leftrightarrow 2$ suffice. Indices identify positions; record identities move during swaps.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Decrease, indexed deletion, and equal priorities

Decreasing a max-heap key can violate only child edges, so repair downward. For deletion at position $i$, replace it with the last record and decrease size; deleting the old last position simply decreases size.

Let replacement key be $z$. If $i > 1$ and $z > A[parent(i)]$, the old parent dominated the deleted key, which dominated its children; consequently $z$ dominates those children too and needs only upward repair. Otherwise the parent edge is valid and only downward repair is needed. Each operation costs worst-case $O(\lg(s+1))$ given a position; searching for an arbitrary key can still cost $\Theta(s)$.

A handle identifies a record. Every swap must update the handle-to-index map, and deletion removes the deleted handle. An array index does not permanently identify a record.



## Sequence numbers have different roles in sorting and queues

Heapsort is unstable: $[2a, 2b, 1c]$ can become $[1c, 2b, 2a]$. Stable ascending sorting can compare $(key, original_index)$, placing later equal records into the suffix first. FIFO ties in a max-priority queue instead compare $(priority, -arrival_index)$. The sequence-number direction differs, and both choices change the comparison and storage specification.

Let $a, b$ arrive in that order, with sequence numbers $1, 2$ and equal keys $5$:

| Purpose | Key for $a$ | Key for $b$ | First maximum extracted | Required order |
|---|---|---|---|---|
| Stable ascending sort | $(5, 1)$ | $(5, 2)$ | $b$, placed in the suffix | Output $a, b$ |
| FIFO priority ties | $(5, -1)$ | $(5, -2)$ | $a$, served immediately | Service $a, b$ |

Ascending sorting places extracted maxima from right to left; a queue serves each extracted maximum immediately. Determine how extraction order maps to the desired order before choosing the sequence-number sign.

## Hiring: distinguish interview and replacement costs

There are $n$ candidates with distinct, comparable ranks; larger is better. After each interview, replace the incumbent exactly when the new candidate is better. Dummy candidate `dummy` ranks below everyone.

```
HIRE(A):
    best = dummy; hires = 0
    for i = 1 to n:
        interview A[i]
        if rank(A[i]) > rank(best):
            best = A[i]
            hire best; hires += 1
    return (best, hires)
```

After iteration $i$, `best` is the best of the first $i$ candidates: initially none have arrived; a better arrival replaces the incumbent, otherwise the current best remains. Thus the final result is the overall best.



## One procedure, three different cost questions

With $X$ hires, interview cost $c_I$ and hiring cost $c_H$, total cost is $nc_I + Xc_H$. Empty input returns `dummy` with $X = 0$.

For $n \geq 1$, increasing ranks give $X = n$ and reverse order gives $X = 1$. Now distinguish three questions:

| Question | Random source or quantifier | Quantity |
|---|---|---|
| Worst case | Maximize over legal inputs | $\max_A X(A) = n$ |
| Average case | Specify an input distribution, e.g. uniform over $n!$ orders | $\mathbb{E}_A[X(A)]$ |
| Randomized procedure | Fix any input and uniformly shuffle it internally | $\mathbb{E}_R[X(A, R)]$ |

A uniform shuffle makes every fixed distinct-rank input a uniform permutation; the processing order must be controllable. Expected bounds permit bad runs: increasing order still has probability $1/n!$. Fewer hires reduce hiring cost, not the number of interviews.

## Indicators turn a count into event probabilities

Fix a finite sample space $\Omega$ with nonnegative outcome probabilities summing to $1$. A random variable has expectation $\mathbb{E}[X] = \sum_{\omega \in \Omega} \Pr(\omega) X(\omega)$.

The indicator $I_E$ is $1$ when event $E$ occurs and $0$ otherwise, so

$$\mathbb{E}[I_E] = 1 \Pr(E) + 0 \Pr(E^c) = \Pr(E).$$

If a target count equals $X = \sum_{i=1}^{m} X_i$, exchanging finite sums gives

$$\mathbb{E}[X] = \sum_{\omega} \Pr(\omega) \sum_{i} X_i(\omega) = \sum_{i} \sum_{\omega} \Pr(\omega) X_i(\omega) = \sum_{i} \mathbb{E}[X_i].$$

This is **linearity of expectation**; the $X_i$ need not be independent. Independence may instead be needed to calculate a probability or factor an intersection probability.

For example, let $X_1 = X_2$ both indicate heads on the same fair toss. Then $\mathbb{E}[X_1 + X_2] = 1$, but $\Pr(X_1 = X_2 = 1) = 1/2$, not $1/4$.



## Three tools: union bound, Markov bound, and tail sums

First identify whether the target is an event probability, an expected count, or the expected maximum. Indicators yield three basic tools.

Since $I_{\cup_i E_i} \le \sum_i I_{E_i}$, taking expectations gives

$$\Pr\left(\bigcup_i E_i\right) \le \sum_i \Pr(E_i).$$

For $X \ge 0$ and $t > 0$, inequality $X \ge t I_{\{X \ge t\}}$ gives Markov's bound

$$\Pr(X \ge t) \le \frac{\mathbb{E}[X]}{t}.$$

## Tail sums: count the levels reached by each outcome

For nonnegative integer $L$, every outcome satisfies $L = \sum_{j \geq 1} I_{\{L \geq j\}}$, hence

$$\mathbb{E}[L] = \sum_{j \geq 1} \Pr(L \geq j).$$

Finite bounds allow finite-sum exchange. Infinite tails follow by monotone limits of nonnegative partial sums, allowing both sides to be infinite. None of these three tools requires independence.

Concrete probability products may still require independent trials. Do not drop those model assumptions merely because the general tool needs no independence.

Example: $L = 0, 1, 3$ with probabilities $1/2, 1/4, 1/4$. Direct expectation gives $0/2 + 1/4 + 3/4 = 1$. The tails are $\Pr(L \geq 1) = 1/2$, $\Pr(L \geq 2) = 1/4$, $\Pr(L \geq 3) = 1/4$; their sum is also $1$.



## Hiring count: the probability of a record at each position

Assume uniformity over all $n!$ distinct-rank permutations, and let $X_i$ indicate hiring at position $i$. Condition on the set of the first $i$ candidates: all $i!$ relative orders remain equally likely, so their best candidate is last with probability $1/i$.

Hiring at $i$ is exactly the event that this candidate is best among the first $i$. Thus

$$\mathbb{E}[X] = \sum_{i=1}^{n} \Pr(X_i = 1) = \sum_{i=1}^{n} \frac{1}{i} = H_n.$$

Since $1/x$ decreases, comparing rectangles with integrals gives

$$\int_1^{n+1} \frac{dx}{x} \leq H_n \leq 1 + \int_1^n \frac{dx}{x}, \qquad n \geq 1.$$

Consequently $\mathbb{E}[X] = \Theta(\ln n)$, but total expected cost remains $nc_I + c_H H_n$: interview cost does not become logarithmic.

## A six-candidate run and its expected-cost comparison

| Position $i$ | Rank | Best after interview | Hire? | Total hires |
|---|---|---|---|---|
| 1 | 2 | 2 | Yes | 1 |
| 2 | 5 | 5 | Yes | 2 |
| 3 | 1 | 5 | No | 2 |
| 4 | 4 | 5 | No | 2 |
| 5 | 6 | 6 | Yes | 3 |
| 6 | 3 | 6 | No | 3 |

Input $[2, 5, 1, 4, 6, 3]$ sets records at positions $1, 2, 5$, making three hires; the uniform mean at this size is $H_6 = 49/20 = 2.45$. Exactly one hire has probability $1/n$; hiring everyone requires the unique increasing permutation, with probability $1/n!$. For empty input define $H_0 = 0$ separately.



## The same method counts inversions and fixed points

Uniformly permute $n$ distinct keys. For positions $i < j$, let $I_{ij}$ indicate $A[i] > A[j]$. Exchanging the two entries bijectively pairs inversion and noninversion outcomes, so $\Pr(I_{ij} = 1) = 1/2$.

$$V = \sum_{i<j} I_{ij}, \qquad \mathbb{E}[V] = \binom{n}{2}\frac{1}{2} = \frac{n(n-1)}{4}.$$

Inversion events sharing positions can be dependent; linearity still applies. When $n = 2$, the count is either $0$ or $1$, although its expectation is $1/2$.

Now uniformly return $n \geq 1$ hats one-to-one to their original owners. Each person receives their own hat with probability $1/n$. The fixed-point count satisfies

$$F = \sum_{i=1}^{n} I_i, \qquad \mathbb{E}[F] = n\frac{1}{n} = 1.$$

For $n = 2$, both hats are correct or both wrong, each with probability $1/2$. Mean one does not imply exactly one correct hat in every outcome. For $n = 0$, directly take $F = 0$.

## Uniform integers by rejecting out-of-range bit patterns

The contract of `RANDOM(a,b)` is an inclusive integer with probability $1/(b-a+1)$ per value, conditionally uniform given prior history. Assume integer $a \le b$ and access to independent fair bits.

Set $m = b - a + 1$. If $m = 1$, return $a$. Otherwise let $k = \lceil \lg m \rceil$, repeatedly read $k$ bits as $z \in \{0, \ldots, 2^k - 1\}$, and return $a + z$ when $z < m$. Reject other patterns and redraw the whole block.

Acceptance probability per round is $p = m/2^k > 1/2$. For any legal value $v$, its eventual return probability is

$$\sum_{t=0}^{\infty} (1-p)^t \frac{1}{2^k} = \frac{1}{p2^k} = \frac{1}{m}.$$

Expected rounds are $1/p < 2$, giving $O(\lg m)$ expected bit cost. Termination is almost sure; there is no fixed worst-case retry limit.

Taking a remainder directly can bias the result: eight three-bit patterns reduced modulo $3$ give preimage counts $3, 3, 2$ for remainders $0, 1, 2$. The pseudorandom generator and bounded-integer sampling procedure are distinct parts of an implementation.



## In-place shuffling fixes one more position per round

Select uniformly from the unfixed suffix and swap the selected record into position $i$. Distinguish record identities even when keys coincide.

```
SHUFFLE(A):
    for i = 1 to n:
        j = RANDOM(i,n)
        swap(A[i],A[j])
```

After round $i$, later swaps never touch $A[1..i]$. Empty input has one empty output; a singleton swaps only with itself. Under unit-cost integer sampling, time is $\Theta(n)$ and extra space is $O(1)$; bit-sampling cost is separate.

## Uniform shuffling: extend a prescribed prefix

Claim: any prescribed sequence of $i-1$ distinct identities occupies $A[1..i-1]$ with probability $(n-i+1)!/n!$. For $i=1$, the empty prefix has probability one.

Conditioned on that prefix, the next desired identity occupies one of the $n-i+1$ remaining positions. Conditional uniform sampling selects it with probability $1/(n-i+1)$, so

$$\frac{(n-i+1)!}{n!}\frac{1}{n-i+1}=\frac{(n-i)!}{n!}.$$

Induction gives probability $0!/n!=1/n!$ for every complete permutation. For $n=4$ and target $[C,D,A,B]$:

| Fixed length | $0$ | $1$ | $2$ | $3$ | $4$ |
|---|---|---|---|---|---|
| Prescribed prefix | empty | $C$ | $C,D$ | $C,D,A$ | $C,D,A,B$ |
| Probability | $1$ | $1/4$ | $1/12$ | $1/24$ | $1/24$ |

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## Four-record shuffle: track the fixed prefix at every round

Start with $[A,B,C,D]$ and draw $j=3,4,3,4$ in successive rounds. Letters identify records rather than comparison keys.

| Round $i$ | Legal indices | Drawn $j$ | Full array after swap | Fixed prefix |
|---|---|---|---|---|
| 1 | $1..4$ | 3 | $[C,B,A,D]$ | $C$ |
| 2 | $2..4$ | 4 | $[C,D,A,B]$ | $C,D$ |
| 3 | $3..4$ | 3 | $[C,D,A,B]$ | $C,D,A$ |
| 4 | $4$ | 4 | $[C,D,A,B]$ | $C,D,A,B$ |

This path has probability $(1/4)(1/3)(1/2)(1)=1/24$. Every prescribed output uniquely determines a legal path: locate its next identity in the remaining suffix and swap it into position $i$.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## The fixed prefix never moves again

Start [A,B,C,D]: select only from the unfixed suffix

| | Pos 1 | Pos 2 | Pos 3 | Pos 4 | Round probability |
|---|---|---|---|---|---|
| i=1, j=3 | C | B | A | D | 1/4 |
| i=2, j=4 | C | D | A | B | 1/3 |
| i=3, j=3 | C | D | A | B | 1/2 |
| i=4, j=4 | C | D | A | B | 1 |

Green cells stay fixed. This path has probability 1/4·1/3·1/2·1=1/24.

Later rounds touch only unfixed positions, so the prefix remains fixed. A trace checks the procedure; the preceding joint invariant proves general uniformity.



## Uniform positions do not guarantee a uniform permutation

A uniform cyclic shift puts each record in every position with probability $1/n$, yet for $n \geq 3$ it produces only $n$ outputs rather than $n!$. Uniform marginals do not imply a uniform joint permutation.

Another incorrect procedure runs $i = 1..n$, drawing $j$ from all of $1..n$ each round before swapping. For $n = 3$, its $3^3 = 27$ random paths are equally likely. Uniformity over six permutations would require $27/6$ paths per output, impossible for an integer path count.

This procedure can move earlier positions again, so the fixed-prefix proof no longer applies. A demonstrated bias is stronger than saying the shuffle looks insufficiently random.

Running only the first $m$ rounds of the correct shuffle, $0 \leq m \leq n$, gives each ordered sample probability $(n-m)!/n!$. Each unordered $m$-subset has $m!$ orders, hence probability $1/\binom{n}{m}$. This saves later swaps, but constructing an indexable array can still cost $\Theta(n)$.

## Random priorities: collision handling is part of uniformity

Give $n \geq 1$ records independent uniform priorities $P_i \in \{1, \ldots, M\}$ for integer $M \geq n$, and sort by priority. Conditioned on all priorities being distinct, exchanging record labels bijectively maps orders while preserving probability. Thus all $n!$ relative orders are equally likely.

Each pair collides with probability $1/M$. The union bound gives

$$\Pr(\exists i < j : P_i = P_j) \leq \frac{\binom{n}{2}}{M}.$$

With $M = n^3$ and $n \geq 2$, this is $(n-1)/(2n^2) < 1/(2n)$. Rare collisions are still possible.

Keeping original order on ties biases the unconditional output. For $n = 2$, the first record precedes the second with probability $\Pr(P_1 \leq P_2) = (M+1)/(2M) > 1/2$.

One repair redraws the entire set after any collision and accepts only distinct priorities. For this $M$, acceptance probability has a positive constant lower bound, so expected rounds are constant. Alternatively, uniformly shuffle each tie group independently. Comparison-based priority sorting takes $\Theta(n \lg n)$ time and $O(n)$ priority storage; redraws must also detect collisions, which is not free.



## After randomization, specify correctness and what is controllable

`RANDOMIZED-HIRE` uniformly shuffles, then runs `HIRE`. Every fixed distinct-rank input becomes a uniform permutation, giving $H_n$ expected hires and expected cost $nc_I + c_H H_n$. Another $\Theta(n)$ shuffle operations must be accounted for separately from monetary hiring costs.

If candidates arrive externally and cannot be reordered, the algorithm lacks this control. The $H_n$ analysis still requires a uniform-arrival assumption; an unavailable shuffle cannot remove it.

For the shuffle and randomized-pivot sort in this chapter, cost is random while every returned result satisfies its specification. These are Las Vegas algorithms. Monte Carlo algorithms allow a small error probability, which must be stated with its input quantifiers. Expected time alone says nothing about correctness probability.

The later online-selection strategy has a different objective: it follows its selection rule even when the chosen candidate is not globally best. Its success probability needs a separate model and proof.

## Quicksort partitions, then sorts both sides

Use the 1-based closed interval $A[p..r]$. Partitioning around a pivot establishes order between regions; it does not sort either region internally.

```
QUICKSORT(A,p,r):
    if p >= r: return
    q = PARTITION-LAST(A,p,r)
    QUICKSORT(A,p,q-1)
    QUICKSORT(A,q+1,r)
```

An empty interval ($p > r$) or singleton ($p = r$) returns immediately. Partition returns the pivot position $q$, and both recursive intervals **exclude** $q$, so their lengths strictly decrease. Correctness and termination depend on this return contract.



## Last-position partition: save the key, move whole records

The last-position partition is given below. Save pivot key $x$ first. Swaps move complete records, keeping every associated field with its key.

```
PARTITION-LAST(A,p,r):
    x = key(A[r]); i = p-1
    for j = p to r-1:
        if key(A[j]) <= x:
            i += 1
            swap(A[i],A[j])
    swap(A[i+1],A[r])
    return i+1
```

## Loop boundaries: i ends the small region; j is next

The pivot stays at A[r]. Use **1-based indices and the inclusive interval A[p..r]**. The boundary i=p−1 means that the ≤ region is empty.

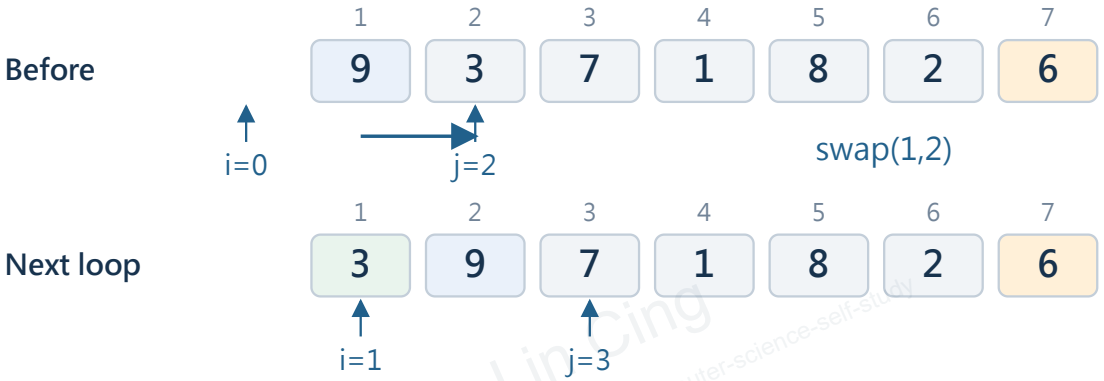
| Region | Known relation | Initially i=p−1, j=p |
|---|---|---|
| A[p..i] | ≤ pivot | Empty |
| A[i+1..j−1] | > pivot | Empty |
| A[j..r−1] | Unclassified | All non-pivot keys |
| A[r] | Pivot | Fixed until the final exchange |

**Diagram key:** green ≤, blue >, gray unknown, orange pivot. Both transition diagrams use [9,3,7,1,8,2,6] with pivot 6.



## A small key: exchange, then advance both boundaries

3 ≤ 6: advance i, then exchange A[1] and A[2]

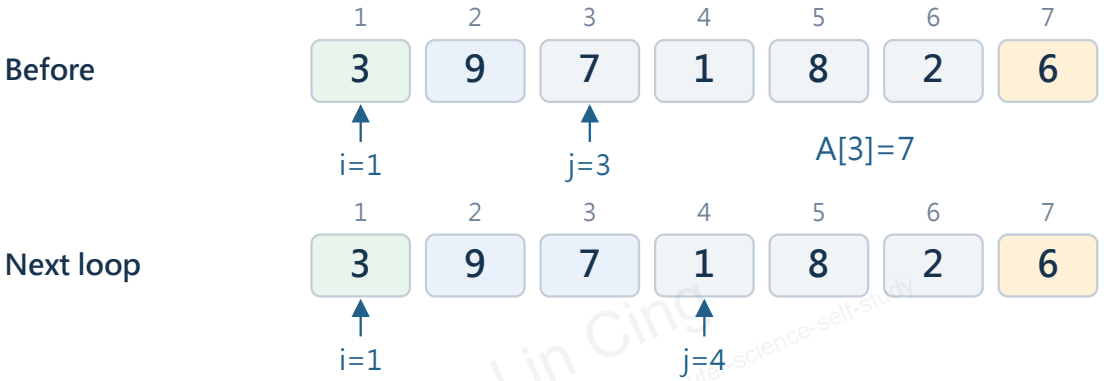



3 joins ≤; displaced 9 stays in >. Pivot 6 remains at index 7.

For A[2]=3, advance i to 1 and exchange A[1],A[2]. The key 3 joins ≤; displaced 9 stays in >. Finally advance j. In general i may equal j, giving a harmless self-exchange.

## A large key: leave values fixed and advance j

7 > 6: no exchange; keep i and advance j



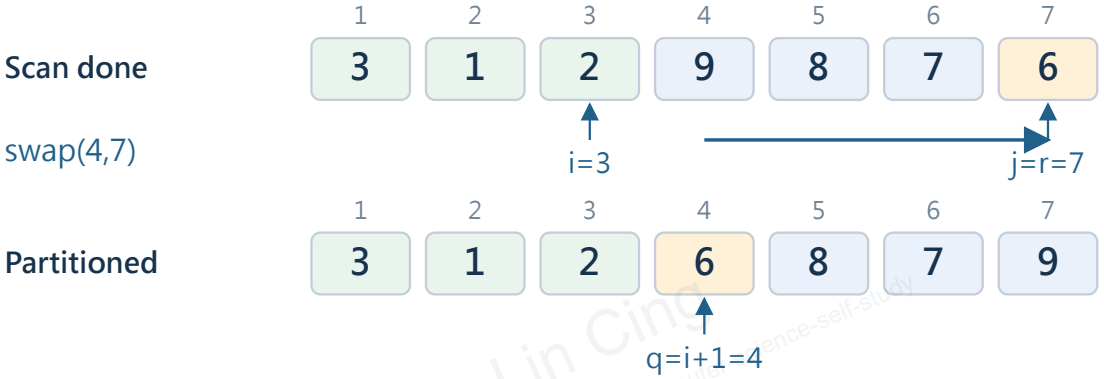

7 moves from unknown to >. Values stay fixed; the boundary moves.

Since A[3]=7>6, leave i and the array unchanged. Advancing j adds 7 to the > region. **The values stay fixed; our classification changes.**



## The final exchange establishes the pivot position q

Only after scanning: place the pivot between the two regions




≤6: [3,1,2] | pivot 6 | >6: [8,7,9]; the sides remain unsorted.

The unknown length r−j decreases once per loop, proving termination. At j=r, exchange A[i+1],A[r] and return q=i+1. The left region is ≤pivot and the right region >pivot; neither side is necessarily sorted.

## From the partition invariant to sorting correctness

1. **Initialization:** i=p−1,j=p makes both classified regions empty.
2. **Maintenance:** a small key expands $\leq$ after an exchange; a large key expands >. Either branch removes one unknown key and leaves the pivot fixed.
3. **Partition termination:** no unknown keys remain. The final exchange places the pivot at q. Exchanges preserve the input multiset.
4. **Recursive induction:** empty and singleton intervals are sorted. Both children exclude q and are strictly shorter. Sorting them and adjoining the pivot therefore sorts the whole interval.

The ordering relation alone is insufficient: the proof also needs **multiset preservation, strictly smaller recursive intervals**, and sorted children upon return.



## Complete partition trace: every test and exchange

Input is $A = [9, 3, 7, 1, 8, 2, 6]$ with $p = 1, r = 7$. Save last key $x = 6$ and initialize $i = 0$. The table gives $i$ after each test.

| Scan index $j$ | Test | $i$ | Full array after action |
|---|---|---|---|
| 1 | $9 > 6$, no swap | 0 | $[9, 3, 7, 1, 8, 2, 6]$ |
| 2 | $3 \leq 6$, swap $1, 2$ | 1 | $[3, 9, 7, 1, 8, 2, 6]$ |
| 3 | $7 > 6$, no swap | 1 | $[3, 9, 7, 1, 8, 2, 6]$ |
| 4 | $1 \leq 6$, swap $2, 4$ | 2 | $[3, 1, 7, 9, 8, 2, 6]$ |
| 5 | $8 > 6$, no swap | 2 | $[3, 1, 7, 9, 8, 2, 6]$ |
| 6 | $2 \leq 6$, swap $3, 6$ | 3 | $[3, 1, 2, 9, 8, 7, 6]$ |
| End of scan | Swap $4, 7$ | 3 | $[3, 1, 2, 6, 8, 7, 9]$ |

Return $q = 4$; both sides remain unsorted. Six key comparisons; $\Theta(r - p + 1)$ time and $O(1)$ extra space.

## Worst-case time: a family attaining quadratic work

Sorted distinct input with a fixed final pivot always chooses the maximum. The two child lengths are n−1 and 0.

| Current length | Key comparisons | Next nonempty length |
|---:|---:|---:|
| n | n−1 | n−1 |
| n−1 | n−2 | n−2 |
| … | … | … |
| 2 | 1 | 1 |

The sum is $n(n-1)/2$, proving a worst-case Ω(n²) lower bound for an entire input family, rather than extrapolating from one measurement.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## The quadratic upper bound must cover every split

Let C(n) be the worst comparison count. Child sizes satisfy a+b=n−1. Induction on smaller sizes gives

$$C(a)+C(b)+(n-1)\le \frac{a(a-1)}{2}+\frac{b(b-1)}{2}+n-1=\frac{n(n-1)}{2}-ab\le \frac{n(n-1)}{2}.$$

With C(0)=C(1)=0, this covers all splits. Equality holds when a=0 or b=0. Each pivot leaves recursion, so there are O(n) calls. Total time is O(n+C); the matching family proves **worst-case Θ(n²)**.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Balanced splits: count the comparison work by level

Take n=15 and always choose the true median. Count only Lomuto key comparisons; singletons do not call partition.

| Depth d | Subproblems | Size each | Comparisons at level |
|---|---|---|---|
| 0 | 1 | 15 | 14 |
| 1 | 2 | 7 | 2×6=12 |
| 2 | 4 | 3 | 4×2=8 |
| 3 | 8 | 1 | 0 |

Total: 34 comparisons, versus 105 for endpoint splits on 15 keys. Partition still scans linearly; the difference is how much work survives at each depth.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A fixed split ratio also gives logarithmic depth

If the larger child always has size at most ρ times the parent size, for a fixed ρ<1, the longest branch has O(log n) levels. Intervals at one depth are disjoint, giving at most n scanned keys per level and O(n log n) total work. A fixed 1:9 split suffices.

For perfectly balanced $n = 2^k - 1$, depth d has $2^d$ subproblems of length $2^{k-d} - 1$. Its comparison work is

$$2^d(2^{k-d} - 2) = n + 1 - 2^{d+1}, \qquad 0 \le d \le k - 2.$$

The first roughly k/2 levels each require Θ(n) comparisons, proving Ω(n log n) for that execution and hence Θ(n log n) overall. This lower bound counts actual recursive work; a comparison-sorting worst-input lower bound does not apply to every individual input.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Duplicates: a random pivot cannot repair all-equal input

For $[5, 5, 5, 5, 5]$, every `<= x` test succeeds, so every partition returns $q = r$. Successive sizes $5, 4, 3, 2$ cost $4 + 3 + 2 + 1 = 10$ comparisons; for $n$ records the total is $n(n-1)/2$.

A randomly chosen pivot is still $5$ and cannot change this path. The later expected $\Theta(n \lg n)$ analysis of the two-way version therefore assumes **distinct keys**. The order-statistics section gives three-way partitioning into keys less than, equal to, and greater than the pivot. Only the strict regions recurse. All-equal input then finishes in one $\Theta(n)$ partition.

Stability is a separate guarantee. Input $[2_a, 2_b, 1_c]$ with last pivot $1$ becomes $[1_c, 2_b, 2_a]$ after the final swap. The two key-$2$ records have reversed order. Random pivot selection does not make this swapping implementation stable.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Recurse on the smaller side to bound stack space

The original two-recursion procedure can retain $\Theta(n)$ stack frames. In-place partitioning alone does not give constant-space quicksort. Recurse only on the smaller side and handle the larger side in the current frame's loop.

```
QUICKSORT-SMALL-STACK(A,p,r):
    while p < r:
        q = PARTITION-LAST(A,p,r)
        if q-p < r-q:
            QUICKSORT-SMALL-STACK(A,p,q-1)
            p = q+1
        else:
            QUICKSORT-SMALL-STACK(A,q+1,r)
            r = q-1
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Smaller-side recursion bounds the live call chain

Side lengths satisfy $a + b = n - 1$, so the actual recursive call has size at most $\lfloor (n-1)/2 \rfloor$. Sizes at least halve along the chain of unreturned calls, yielding worst-case $O(\lg(n+1))$ stack space.

If every pivot is largest, the recursive side is empty but the loop still partitions sizes $n, n-1, \ldots, 2$. Stack space is then constant while time remains $\Theta(n^2)$. Analyze these two resources separately.

Median pivots: a chain of live calls

Nested frames




15 → 7 → 3 → 1: four simultaneous frames

Largest pivots: successive loop iterations

One working frame




Each empty recursive call returns immediately; the larger side stays in this frame.



## Randomization bounds expectation for each fixed input

At each partition, draw integer $k$ uniformly from closed interval $p..r$, swap $A[k]$ with $A[r]$, then use the same last-position partition. Future choices must remain uniform conditional on their current states.

For distinct keys, every local rank has pivot probability $1/n$, whatever the current arrangement. Expectation is over **internal algorithm choices**; no uniform-input assumption is required.

Every fixed input can still encounter a path that repeatedly chooses an endpoint, costing $\Theta(n^2)$. An expectation is neither a per-execution upper bound nor an unproved tail-probability bound.

Separate mathematical uniform sampling from its implementation. A fixed seed reproduces a pseudorandom sequence; merely calling random does not prove uniformity. Bounded-integer sampling can use the rejection procedure developed earlier in this chapter.

## A pair compares exactly when an interval endpoint is first

Name distinct keys in increasing order $z_1 < \cdots < z_n$. Let $X_{ij}$ indicate whether $z_i, z_j$ are directly compared. They compare at most once: one must be the pivot, which then leaves recursion.

Consider $Z_{ij} = \{z_i, \ldots, z_j\}$. An outside pivot keeps this entire interval on one side. If the first pivot inside it is endpoint $z_i$ or $z_j$, the pair compares immediately. If it is an interior key, the endpoints enter different recursive regions and never meet again.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Outside pivots preserve the entire rank interval

**Sorted ranks for analysis: $Z_{2,5}$ = {2,3,4,5}; not an array snapshot**




Pivot 7 first: ranks 2..5 remain together in its left child; $X_{2,5}$ is undecided.

**First within interval: endpoint 2 (or 5)**




This partition compares 2 and 5 → $X_{2,5}$=1

**First within interval: interior 3 (or 4)**




2 left, 5 right → never compare, $X_{2,5}$=0

Four interval keys are equally likely to be first; two are endpoints. P($X_{2,5}$=1)=2/4=1/2.

Thus $X_{ij} = 1$ exactly when the first pivot selected from $Z_{ij}$ is one of its endpoints. It need not be the first pivot of the entire execution.

## From four first choices to the general comparison probability

For $z_2, z_5$, inspect only the first pivot from $Z_{2,5} = \{z_2, z_3, z_4, z_5\}$.

| First pivot inside the interval | What happens to $z_2, z_5$ | $X_{2,5}$ |
|---|---|---|
| $z_2$ | Compare immediately | 1 |
| $z_3$ | Separate forever | 0 |
| $z_4$ | Separate forever | 0 |
| $z_5$ | Compare immediately | 1 |

Each key has probability $1/4$ of being first inside the interval, so the comparison probability is $1/2$. Generally, for interval length $m = j - i + 1$,

$$\Pr(X_{ij} = 1) = \frac{2}{m} = \frac{2}{j - i + 1}.$$



## Why the first pivot inside an interval is uniform

Prove uniformity rather than assume it. Fix an interval $Z$ of $m$ keys and a designated $z \in Z$. Induct on the size $N$ of the current subproblem containing $Z$. If $N = m$, the first pivot is uniform, giving probability $1/m$. For $N > m$, choosing $z$ immediately has probability $1/N$; choosing another key of $Z$ fails.

An outside pivot leaves all of $Z$ in a strictly smaller subproblem. Induction gives designated-first probability $1/m$ there. Summing over the $N - m$ outside keys yields

$$\frac{1}{N} + \frac{N - m}{N} \frac{1}{m} = \frac{1}{m}.$$

The first pivot inside the interval is therefore uniform; the denominator is $m$, not original input size $n$. Other recursive branches may execute in between but do not move or compare these keys.

## Expected comparisons: sum over rank-interval lengths

Total key comparisons are $C = \sum_{1 \le i < j \le n} X_{ij}$. Linearity of expectation requires no independence. There are $n + 1 - k$ rank intervals of length $k = j - i + 1$, so

$$\begin{aligned}
\mathbb{E}[C] &= \sum_{i<j} \frac{2}{j-i+1} = 2\sum_{k=2}^{n} \frac{n+1-k}{k} \\
&= 2(n+1)\sum_{k=2}^{n} \frac{1}{k} - 2(n-1) \\
&= 2(n+1)H_n - 4n, \qquad H_n = \sum_{k=1}^{n} \frac{1}{k}.
\end{aligned}$$

Bounds $\ln(n+1) \le H_n \le 1 + \ln n$ give expected $\Theta(n \lg n)$ comparisons. Scanning work is proportional to these comparisons, with $O(n)$ additional call overhead, so expected time is also $\Theta(n \lg n)$.

The exact formula assumes distinct keys, conditionally uniform pivots, and one <= test per nonpivot key. It excludes index tests, swaps, and random-number generation. Comparison events are dependent; independent-variable variance or tail formulas cannot be imported without further proof.



## A first-position pivot has its own invariant

This single-scan version keeps pivot $x = A[p]$ at the first position until cleanup. The interval is still the closed interval $A[p..r]$.

```
PARTITION-FIRST(A,p,r):
    x = A[p]; i = p
    for j = p+1 to r:
        if key(A[j]) <= key(x):
            i = i+1
            swap(A[i],A[j])
    swap(A[p],A[i])
    return i
```

Before each test, $A[p] = x$, $A[p+1..i] \le x$, $A[i+1..j-1] > x$, and $A[j..r]$ is unknown. Small keys expand the left region; large keys only advance $j$. The unknown length decreases each round. Cleanup places the pivot at $i$; exclude $i$ from both recursive sorts.

## Check both partitions using the same pivot key

Use first-position input $[6, 10, 13, 5, 8, 3, 2, 11]$, pivot $x = 6$, and initial $i = 1$.

| $j$ | Test and action | Array after action |
|---|---|---|
| 2 | $10 > 6$, $i = 1$ | $[6, 10, 13, 5, 8, 3, 2, 11]$ |
| 3 | $13 > 6$, $i = 1$ | Unchanged |
| 4 | $5 \leq 6$, $i = 2$, swap $2, 4$ | $[6, 5, 13, 10, 8, 3, 2, 11]$ |
| 5 | $8 > 6$, $i = 2$ | Unchanged |
| 6 | $3 \leq 6$, $i = 3$, swap $3, 6$ | $[6, 5, 3, 10, 8, 13, 2, 11]$ |
| 7 | $2 \leq 6$, $i = 4$, swap $4, 7$ | $[6, 5, 3, 2, 8, 13, 10, 11]$ |
| 8 | $11 > 6$, $i = 4$ | Unchanged |
| End | Swap $1, 4$, return $4$ | $[2, 5, 3, 6, 8, 13, 10, 11]$ |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Check the same pivot in both variants

### Same pivot key, different internal orders

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| First pivot | 2 | 5 | 3 | 6 | 8 | 13 | 10 | 11 |
| Last pivot | 5 | 3 | 2 | 6 | 8 | 10 | 13 | 11 |

Both: q=4, pivot=6; left <=6, right >6. Regions need not be sorted.

To choose the same key $6$ in the last-position version, first exchange the endpoints, then partition $[11, 10, 13, 5, 8, 3, 2, 6]$. The result can be $[5, 3, 2, 6, 8, 10, 13, 11]$. Both place $6$ at position $4$ and make $7$ scan-key comparisons; their internal region orders may differ.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Compare contracts, boundaries, and return values

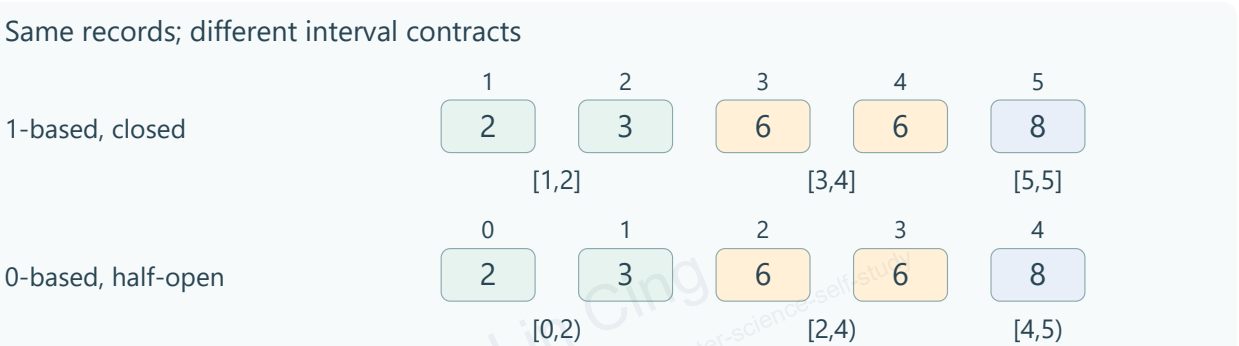
| Version | Pivot and initial boundaries | Return value and recursion |
|---|---|---|
| CLRS last-position single scan | $x = A[r], i = p - 1, j = p$ | One pivot index $q$; recurse on $[p, q - 1], [q + 1, r]$ |
| MIT 2005 first-position single scan | $x = A[p], i = p, j = p + 1$ | One pivot index $q$; exclude $q$ again |
| First-position inward scan with final placement | $x = A[p]$; both scans stop on equality | Exchange pivot after crossing; return its position $q$ |
| This chapter's three-way partition | Save pivot key $x$; $lt = i = p$, $gt = r$ | Equal closed interval $[lt, gt]$; exclude that whole interval |

Inward scanning still leaves two return contracts to distinguish. Classical Hoare partition returns a split boundary without guaranteeing the pivot's final position. Another inward-scan variant finally exchanges the pivot and returns its position. Their recursive intervals differ; the name Hoare alone is insufficient. The inward-placement code below specifies the latter procedure.



## Two interval contracts for the same records

The two highlighted keys are the same records: closed $[3, 4]$ in 1-based indexing, half-open $[2, 4)$ in 0-based indexing.




Convert closed $[p, r]$ to half-open $[p - 1, r)$: exclude the right endpoint. Likewise $[p, q - 1]$ becomes $[p - 1, q - 1)$. Empty intervals have length zero.

## Inward scanning with final placement stops on equal keys

Use closed interval $p \le r$ with the first record as pivot. Evaluate and left to right with short-circuiting: check the boundary before reading an array entry.

```
PARTITION-INWARD-PLACE(A,p,r):
    x = key(A[p]); i = p+1; j = r
    while i <= j:
        while i <= j and key(A[i]) < x: i += 1
        while i <= j and key(A[j]) > x: j -= 1
        if i >= j: break
        swap(A[i],A[j])
        i += 1; j -= 1
    swap(A[p],A[j])
    return j
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## Inward scanning: invariant and complete exchanges

Classified regions satisfy $A[p+1..i-1] \le x$ and $A[j+1..r] \ge x$. Scans skip strictly smaller or strictly larger keys. Before crossing, exchange the stopped records and advance both indices, so equal keys cannot stall progress. The unknown region shrinks; finally exchange the pivot into $j$. Recurse on $[p, j-1]$ and $[j+1, r]$.

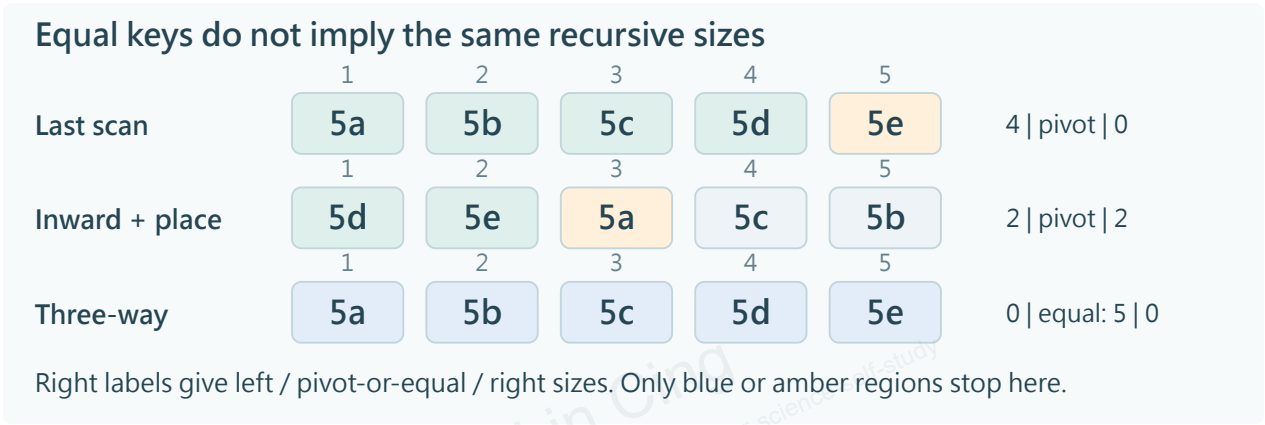
Input $[6, 10, 13, 5, 8, 3, 2, 11]$, $x = 6$. Scan stops are recorded before the exchange.

| Stopped $(i, j)$ | Exchange | Complete array afterward |
|---|---|---|
| $(2, 7)$ | $2, 7$ | $[6, 2, 13, 5, 8, 3, 10, 11]$ |
| $(3, 6)$ | $3, 6$ | $[6, 2, 3, 5, 8, 13, 10, 11]$ |
| $(5, 4)$ crossed | Final $1, 4$ | $[5, 2, 3, 6, 8, 13, 10, 11]$ |

Return $q = 4$. Left keys are $\le 6$ and right keys are $\ge 6$; neither side needs to be internally sorted.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# Five equal keys produce three different partition outcomes




Letters identify records. Inward placement swaps $2, 5$, then $3, 4$, then $1, 3$. For all-equal inputs, its nearly halved sides give $\Theta(n \lg n)$ time, versus $\Theta(n^2)$ for last-position scanning and $\Theta(n)$ for three-way partitioning. These are input-family bounds; inward scanning can still be quadratic on other inputs.



# One classification can require two key comparisons

The exact formula counts executions of `key(A[j]) <= x` in the last-position single scan: one per nonpivot key. A three-way version that tests `< x` and then, on failure, `> x` can use two Boolean comparisons per classification, including for an equal key.

For keys $[1, 4, 3]$ classified against pivot value $x = 3$, key $1$ needs one test; keys $4, 3$ need two each, totaling five rather than three. In-place three-way swapping may change processing order, but its branch-test rule must still be counted.

The data-structures chapter uses 0-based half-open $[l, r)$ and equal region $[lt, gt)$. This chapter uses 1-based closed $[p, r]$ and equal region $[lt, gt]$. Both exclude the entire equal region, but boundaries and exact comparison constants require separate verification.

With distinct keys and conditionally uniform pivots, three-way quicksort still has expected $\Theta(n \lg n)$ time: each level performs at most constant-factor linear classification work, with the same recursive sizes. This asymptotic conclusion does not make its Boolean count exactly $2(n + 1)H_n - 4n$.

## Four ranks: pair probabilities differ from run costs

Name four distinct keys $z_1, z_2, z_3, z_4$ by increasing rank; their input order is arbitrary.

| Rank pairs | Interval length | Direct-comparison probability |
|---|---|---|
| $(1,2), (2,3), (3,4)$ | 2 | $1$ each |
| $(1,3), (2,4)$ | 3 | $2/3$ each |
| $(1,4)$ | 4 | $1/2$ |

Thus $\mathbb{E}[C] = 3 + 4/3 + 1/2 = 29/6$. Substituting $H_4 = 25/12$ gives the same $2 \cdot 5 \cdot 25/12 - 16 = 29/6$.

A run uses an integer number of comparisons. A first pivot of rank $2$ or $3$ gives four comparisons with probability $1/2$. An endpoint first, then the median of the remaining three, gives five with probability $(1/2)(1/3) = 1/6$. Remaining runs use six, with probability $1/3$. Hence $4/2 + 5/6 + 6/3 = 29/6$, independently checking the expected run cost.

Adjacent ranks always compare, but not every pair does. Average $29/6$ does not mean any run performs a fractional comparison.



## Counting sort assigns an output block to each key

Preconditions: integer $K \geq 0$ and keys in $0..K$. Input/output positions are $1..n$. Key-indexed storage must support $K + 1$ entries, and each counter must represent values $0..n$.

```
COUNTING-SORT(A,K):
    C[0..K] = 0
    for j = 1 to n: C[key(A[j])] += 1
    for v = 1 to K: C[v] += C[v-1]
    for j = n downto 1:
        v = key(A[j])
        B[C[v]] = A[j]
        C[v] -= 1
    return B
```

After counting, $C[v]$ is the frequency of key $v$. After prefix accumulation, it counts records with keys $\leq v$: the **last position** of that key's output block. Placement consumes the counters; copy the unconsumed prefix table to $P$ first if range queries will be needed.

## Ending-position placement: trace every write

Take $A = [2_a, 0_b, 2_c, 1_d, 2_e]$ and $K = 2$. Frequencies of keys $0, 1, 2$ are $[1, 1, 3]$, with prefix ending positions $[1, 2, 5]$. Their output blocks are position $1$, position $2$, and positions $3..5$.

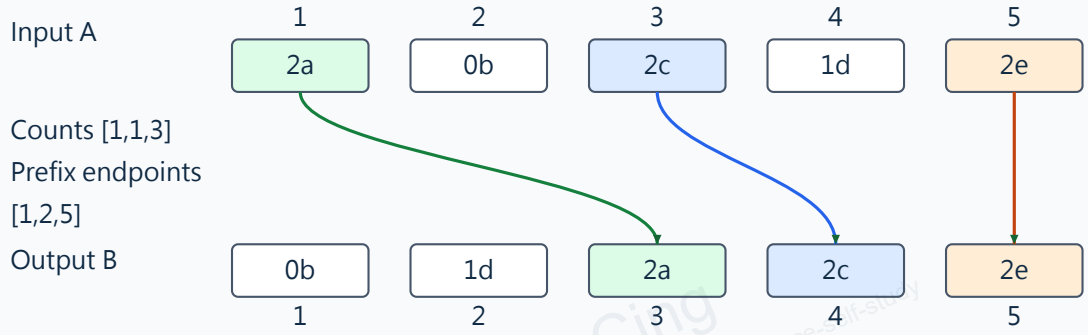
| Scan index $j$ | Record | Position written in $B$ | $C$ after placement |
|---|---|---|---|
| 5 | $2_e$ | 5 | $[1, 2, 4]$ |
| 4 | $1_d$ | 2 | $[1, 1, 4]$ |
| 3 | $2_c$ | 4 | $[1, 1, 3]$ |
| 2 | $0_b$ | 1 | $[0, 1, 3]$ |
| 1 | $2_a$ | 3 | $[0, 1, 2]$ |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Equal keys retain their identity order

### Counting sort: track destinations of equal-key identities




Process e→d→c→b→a backward; positions are 5→2→4→1→3.

Green a, blue c, orange e all have key 2. Output keeps a→c→e.

Output is $[0_b, 1_d, 2_a, 2_c, 2_e]$. Key-2 identities still occur in order $a, c, e$, establishing stability. Bare keys $[0, 1, 2, 2, 2]$ cannot show whether stability holds.

## Placement invariant, stability, and cost

Fix key $v$ and let $L = P[v-1]$ (use $L = 0$ for $v = 0$), $R = P[v]$. Key-$v$ records in the processed input suffix occupy exactly the right end $[C[v]+1, R]$ of their block. Counter $C[v]$ identifies its rightmost unfilled position.

Another key-$v$ record is still unplaced, so $C[v] > L$. Its write is legal and cannot overwrite a placed record; decrementing the counter preserves the invariant. At termination $C[v] = L$, every block contains exactly its frequency of records. Increasing block order proves sortedness and multiset preservation.

If equal-key record $x$ precedes $y$ in input, reverse scanning places $y$ to the right before placing $x$ to its left. This proves stability. Forward scanning with the same ending-position decrement reverses ties; the next section's starting-position increment is the matching forward rule.

Initialization, counting, prefix accumulation, and placement give $\Theta(n + K + 1)$ time and auxiliary space including output. This is linear when $K = O(n)$ and $n \geq 1$; a huge key domain still incurs table cost for a small input.



## Save prefix counts before consuming positions

For $0 \leq a \leq b \leq K$, subtract prefix counts to count the keys in $[a, b]$:

$$Q(a,b) = \begin{cases} P[b], & a = 0, \\ P[b] - P[a-1], & a > 0. \end{cases}$$

Copy $P$ before placement. For the earlier five records:

| Key $v$ | 0 | 1 | 2 |
|---|---|---|---|
| Frequency $F[v]$ | 1 | 1 | 3 |
| Saved prefix $P[v]$ | 1 | 2 | 5 |
| Consumed counter $C[v]$ | 0 | 1 | 2 |

$Q(1,2) = 5 - 1 = 4$; consumed counters give the wrong answer $2 - 0 = 2$. For $a = 0$, use $Q(0,0) = P[0] = 1$, with no negative index.

Preprocessing: $\Theta(n + K + 1)$. Each query: $O(1)$.

## Stable counting sort can also fill from left to right

The ending-position version scans right to left and decrements its counter. Another stable version starts from each key's **first output position** $S[v]$, scans left to right, and increments that position.

Let $F[v]$ be the frequency of key $v$ in domain $0..K$:

```
S[0] = 1
for v = 1 to K:
    S[v] = S[v-1] + F[v-1]
for j = 1 to n:
    v = key(A[j])
    B[S[v]] = A[j]
    S[v] = S[v] + 1
```

Initially $S[v] = 1 + \sum_{u<v} F[u]$, the first position of that key's block. Empty key blocks may share a start position because no records are written for them.



## Trace all eight records through starting positions

Take $A = [3_a, 1_b, 4_c, 2_d, 6_e, 1_f, 2_g, 2_h]$ and $K = 6$. Frequencies for keys $0..6$ are $[0, 2, 3, 1, 1, 0, 1]$, giving initial $S = [1, 1, 3, 6, 7, 8, 8]$.

| $j$ | Record | Output position | Next position for its key |
|---|---|---|---|
| 1 | $3_a$ | 6 | $S[3] = 7$ |
| 2 | $1_b$ | 1 | $S[1] = 2$ |
| 3 | $4_c$ | 7 | $S[4] = 8$ |
| 4 | $2_d$ | 3 | $S[2] = 4$ |

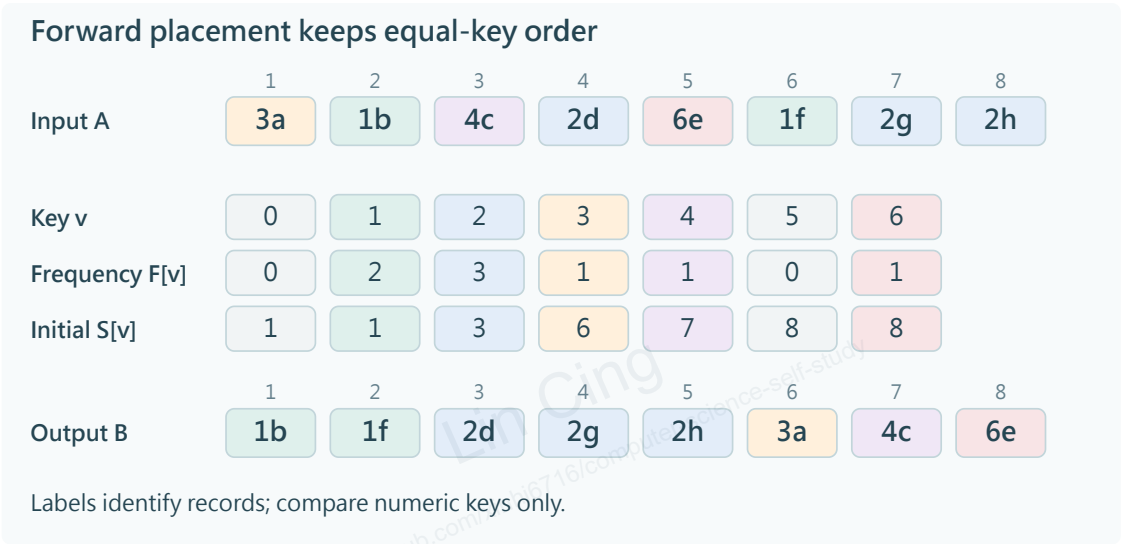
## Finish the eight writes and interpret the final pointers

| $j$ | Record | Output position | Next position for its key |
|---|---|---|---|
| 5 | $6_e$ | 8 | $S[6] = 9$ |
| 6 | $1_f$ | 2 | $S[1] = 3$ |
| 7 | $2_g$ | 4 | $S[2] = 5$ |
| 8 | $2_h$ | 5 | $S[2] = 6$ |

The result is $[1_b, 1_f, 2_d, 2_g, 2_h, 3_a, 4_c, 6_e]$. A final pointer may be one past its key block: it denotes the next unused position, not a position that will necessarily be accessed.



## Equal-key records fill their block in input order




$1_b$ takes position 1 before $1_f$ takes position 2. Records $2_d, 2_g, 2_h$ take positions 3,4,5 in that order. Color connects each equal-key block; identity labels check stability.

## The filled prefix and next-free-position invariant

Let $S_0[v]$ be the unconsumed start and let $c_j(v)$ count key-$v$ records in the already-scanned first $j-1$ inputs. At each iteration's start,

$$S[v] = S_0[v] + c_j(v).$$

Block $[S_0[v], S[v]-1]$ contains exactly the scanned key-$v$ records in input order. Different key blocks do not overlap.

Initially $c_1(v) = 0$ and all filled prefixes are empty. On reading another key-$v$ record, $c_j(v) < F[v]$ because that record is still unplaced; the write is inside its legal block. Writing its next free position and incrementing $S[v]$ preserves records and order, without changing other blocks.

At termination $c_{n+1}(v) = F[v]$, so every key block is full and blocks occur in increasing key order. The result is correct and stable. Counting, start construction, and scanning take $\Theta(n+K+1)$ time and $\Theta(n+K+1)$ auxiliary space including output.

**Pair scan direction with position meaning:** forward/start/increment, or backward/end/decrement. Changing only the scan direction may preserve sorted keys while reversing equal-key identities.



## LSD radix sorting preserves lower-digit order within ties

Let every key be a fixed $d \geq 1$-digit nonnegative integer in base $r \geq 2$, padded with leading zeros. LSD processes digit positions $1, 2, \ldots, d$ from least to most significant, using a **stable** sort at every pass.

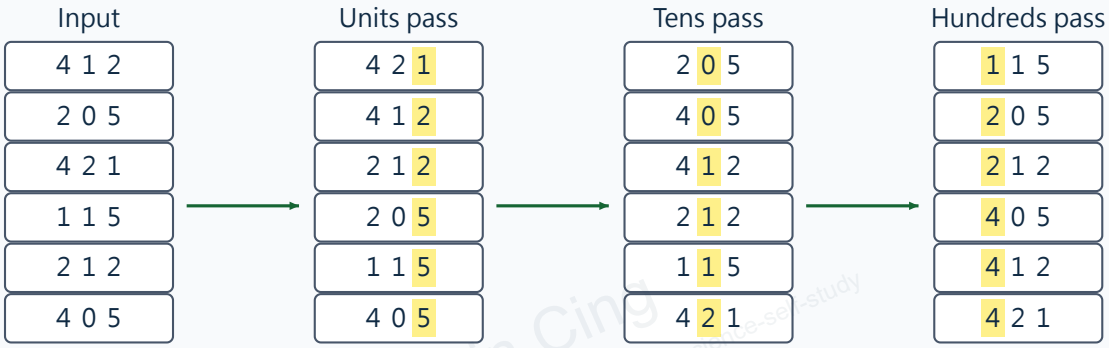
Invariant: after pass $t$, the sequence is sorted by its lowest $t$ digits. The $t = 0$ condition is empty. Pass $t$ groups records by that digit. It determines order between different groups, while stability retains the previous $t-1$-digit order inside a group. Thus the lowest $t$ digits are sorted; $t = d$ gives complete order.

Lower-digit order is retained **within an equal higher-digit group**, not globally across all records. For $[12, 11]$, units sorting gives $[11, 12]$. Tens digits are both $1$; an unstable reversal would incorrectly restore $[12, 11]$.

Sorting the entire sequence by hundreds, tens, then units makes units the final primary key. MSD must recurse within equal higher-digit groups; it is not LSD with its loop simply reversed.

## Three radix passes with every full sequence

### LSD radix: lower digits break higher-digit ties




Yellow marks the current digit. Equal current digits retain the previous column order.

In the hundreds pass, $205$ precedes $212$ because their shared hundreds group retains lower-two-digit order $05 < 12$. Each pass inspects the current digit; the invariant builds full-key order progressively.



## Verify every radix pass

| Digits processed | Complete sequence |
|---|---|
| None | $[412, 205, 421, 115, 212, 405]$ |
| Units | $[421, 412, 212, 205, 115, 405]$ |
| Units and tens | $[205, 405, 412, 212, 115, 421]$ |
| All three | $[115, 205, 212, 405, 412, 421]$ |

In the tens pass, $205, 405$ both have tens digit $0$ and retain their previous order. Records $412, 212, 115$ all have tens digit $1$ and also retain their prior order; their lowest two digits are $12, 12, 15$.

## Account jointly for bit length, radix, and passes

Stable counting sort takes $\Theta(n + r)$ per pass, giving $\Theta(d(n + r))$ over $d$ passes and $\Theta(n + r)$ reusable auxiliary space. Linear time follows directly only for fixed $d$ and $r = O(n)$.

For $b \geq 1$-bit keys and $s$ bits per pass, $1 \leq s \leq b$,

$$r = 2^s, \qquad d = \lceil b/s \rceil, \qquad T = \Theta(\lceil b/s \rceil (n + 2^s)) .$$

Pad a short highest group with zeros. This assumes constant-time key/digit access; integers longer than a machine word require explicit extraction and movement costs.

For $n \geq 2$, choose $s = \min(b, \lfloor \lg n \rfloor)$, so $2^s \leq n$ and time is $O(n\lceil b/s \rceil)$. When $b = O(\lg n)$ this is $O(n)$.

Under a cost of $n + r$ per pass, 32-bit keys with $s = 8$ cost $4(n + 256)$, while $s = 16$ costs $2(n + 65536)$. The latter is smaller exactly when $n > 65024$, with equality at that threshold. Actual allocation, cache, and bandwidth costs lie outside this specified model.



## Distribution is a shared technique, not one algorithm

| Method | Distribution rule | What follows distribution |
|---|---|---|
| LSD radix sort | Digits from least to most significant | Stable full-sequence sort at each pass |
| MSD radix sort | Start with the most significant digit | Recurse on the next digit inside each equal-prefix group |
| Interval bucket sort | Map $x \in [0, 1)$ to $\lfloor nx \rfloor$ | Fully sort each bucket, then concatenate buckets |

All can use buckets, but their **digits, recursion rules, and cost assumptions differ**. Ten digit buckets are not the same as $n$ equal-width intervals. Fully sorting a bucket is not the same procedure as recursively distributing its next digit.

Grouping $[312, 305, 121, 118, 214, 207]$ by hundreds yields $[121, 118]$, $[214, 207]$, $[312, 305]$. This is not yet sorted. MSD processes lower digits within each group; interval bucket sorting uses its specified local sorting algorithm. Concatenating the unsorted groups is incorrect.

## Interval bucket sorting distributes, sorts, and concatenates

For $n \geq 1$ keys $x \in [0, 1)$, create $n$ buckets $B[0..n-1]$ and append each key to $B[\lfloor nx \rfloor]$ in input order. Insertion-sort each bucket, then concatenate by bucket index. Empty input returns immediately. Key $x = 1$ is outside the domain and cannot index $B[n]$.

Correctness needs no distribution assumption: $x \leq y$ implies $\lfloor nx \rfloor \leq \lfloor ny \rfloor$. Different buckets are ordered by disjoint half-open intervals; local sorting orders records within each bucket. Input-order appending plus stable insertion sort also guarantees stability; head insertion may change equal-key order first.

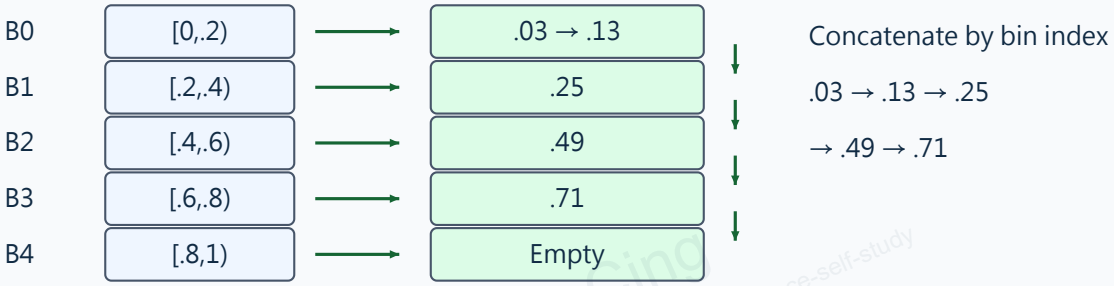
Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Five buckets: interval, local order, and output

### Bucket intervals guarantee cross-bin order

Input .13,.25,.03,.71,.49 → bins 0,1,0,3,2




Blue: half-open bin intervals. Green: sorted contents. No further cross-bin comparisons.

For $n = 5$, input $[.13, .25, .03, .71, .49]$ maps to bucket indices $[0, 1, 0, 3, 2]$. Bucket $0$ becomes $[.03, .13]$; buckets $1, 2, 3$ contain $.25, .49, .71$ respectively; bucket $4$ is empty. Output is $[.03, .13, .25, .49, .71]$.

## Expected linear time uses squared bucket sizes

Let $N_i$ be bucket $i$'s size. Initialization, distribution, and concatenation cost $\Theta(n)$; insertion sorting costs at most a constant times $\sum_i N_i^2$. Thus

$$T \le c_0 n + c_1 \sum_{i=0}^{n-1} N_i^2.$$

Now introduce the input assumption: keys are **mutually independent and uniform** on $[0, 1)$. Indicator $X_{ij}$ records whether input $j$ enters bucket $i$. Then $N_i = \sum_j X_{ij}$, $X_{ij}^2 = X_{ij}$, and

$$\begin{aligned}\mathbb{E}[N_i^2] &= \sum_j \mathbb{E}[X_{ij}] + \sum_{j \ne k} \mathbb{E}[X_{ij} X_{ik}] \\ &= n\frac{1}{n} + n(n-1)\frac{1}{n^2} = 2 - \frac{1}{n}.\end{aligned}$$

Hence $\mathbb{E}[\sum_i N_i^2] = 2n - 1$, giving $\mathbb{E}[T] = O(n)$. Reading the input supplies $\Omega(n)$, so expected time is $\Theta(n)$.

Independence is used for the destinations of different records $j, k$. Bucket sizes are dependent because they sum to $n$, but expectation linearity still applies. Knowing $\mathbb{E}[N_i] = 1$ does not imply $\mathbb{E}[N_i^2] = 1$.



## Concentration, dependence, and a genuine worst-case input

If records remain independent but bucket probabilities are $p_i$, with $\sum_i p_i = 1$, the same expansion gives

$$\mathbb{E}\left[\sum_i N_i^2\right] = n + n(n-1)\sum_i p_i^2.$$

The expected upper bound remains linear when $\sum_i p_i^2 = O(1/n)$. Uniform probabilities give $1/n$; concentrating all probability in one bucket makes the squared-size sum $n^2$.

Uniform marginals alone do not suffice. Choose uniform $J \in \{0, \dots, n-1\}$ and set every $x_j = (J + 1/2)/n$. Every record has uniform bucket marginals, yet all enter one bucket, so $\sum_i N_i^2 = n^2$. This refutes a squared-cost estimate; equal-key insertion sorting itself may still be linear.

For an actual quadratic-time family, take $n \ge 2$ and $x_j = (n - j + 1)/(2n^2)$. These keys are distinct, strictly decreasing, and all in bucket $0$. Insertion sort performs $n(n-1)/2$ shifts, establishing worst-case $\Theta(n^2)$ time.

Replacing each local sort by a worst-case $O(m \lg(m+1))$ method gives total worst-case $O(n \lg(n+1))$. Independent uniform inputs retain linear expected time because $m \lg(m+1) = O(m^2)$.

## Order statistics and extrema: specify the target

Inputs contain $n \geq 1$ totally ordered keys. The $k$th smallest value occupies position $k$ in the sorted multiset, for $1 \leq k \leq n$. Repeated occurrences take separate positions but can yield the same answer: ranks $1, 2$ of $[2, 5, 2, 8]$ both return $2$.

Our lower median has rank $\lceil n/2 \rceil$. For even $n$, it is the smaller central value, unlike the statistical convention that averages the two central values. Reject empty inputs and invalid ranks at entry.

A minimum scan initializes from the first record and compares each remaining key with the current minimum. Its invariant is that the current value is the minimum of the processed prefix. It uses $n - 1$ key comparisons; maximum is symmetric.

Distinct-key inputs give a matching comparison lower bound: every nonminimum record must be shown larger than something before it can be eliminated. A comparison can eliminate at most one new minimum candidate. Thus at least $n - 1$ comparisons are needed. Using distinct keys for a worst-case lower bound does not restrict the algorithm to distinct-key inputs.



## Paired extrema: eight records, ten comparisons

Compare each pair first: only its smaller record challenges the minimum, its larger the maximum. Initialize from the first pair; later pairs preserve both extrema of the processed prefix.

Input: $[14, 3, 9, 1, 8, 5, 12, 6]$.

| Pair | Smaller, larger | Minimum | Maximum | Comparisons |
|---|---|---|---|---|
| $14, 3$ | $3, 14$ | 3 | 14 | 1 |
| $9, 1$ | $1, 9$ | 1 | 14 | 4 |
| $8, 5$ | $5, 8$ | 1 | 14 | 7 |
| $12, 6$ | $6, 12$ | 1 | 14 | 10 |

Even $n$: $1 + 3(n - 2)/2$. Odd $n$: initialize from one record, then use $3(n - 1)/2$. Both give $\lceil 3n/2 \rceil - 2$ comparisons (zero for $n = 1$). Ties preserve correct extremal values.

## Why paired extrema achieve the worst-case lower bound

For distinct keys, every record initially qualifies as both a minimum and a maximum candidate. Only one qualification of each type survives, so $2n - 2$ qualifications must be eliminated.

An adversary answers comparisons as follows. Let $U$ mean untested, $H$ maximum-only, $L$ minimum-only, and $M$ neither.

| Compared states | Answer and new eliminations |
|---|---|
| $U, U$ | Choose a larger and a smaller key; eliminate 2 qualifications |
| $H, H$ or $L, L$ | Eliminate one maximum or minimum candidate |
| $H, L$ | Answer $H > L$; eliminate none |
| $U$ versus a classified key | Against $H$, answer $U < H$; against $L$, answer $U > L$; against $M$, choose either consistent direction. Eliminate one $U$ qualification |
| Involving $M$, with no $U$ | Maintain $H > M > L$; for $M, M$ use a linear extension of the existing order. At most one new elimination |

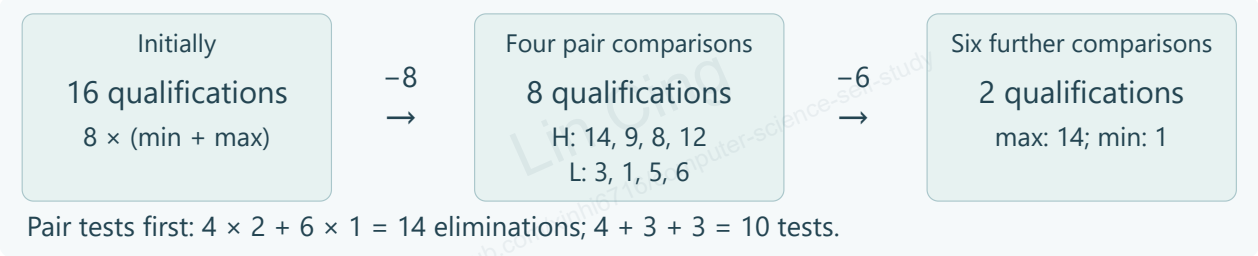


## Count the eliminated candidate qualifications

The answers stay consistent: $H$ keys have no known larger key, $L$ keys no known smaller key, and $U$ keys no constraints. Place $H$ above $M$ above $L$, extending the comparison order within each class; no cycle is introduced.

Only $U, U$ removes two qualifications, at most $\lfloor n/2 \rfloor$ times. Other comparisons remove at most one. Thus

$$2n - 2 \le c + \lfloor n/2 \rfloor, \qquad c \ge \lceil 3n/2 \rceil - 2.$$




Pair tests first: 4 × 2 + 6 × 1 = 14 eliminations; 4 + 3 + 3 = 10 tests.

## A tournament preserves the second-minimum candidates

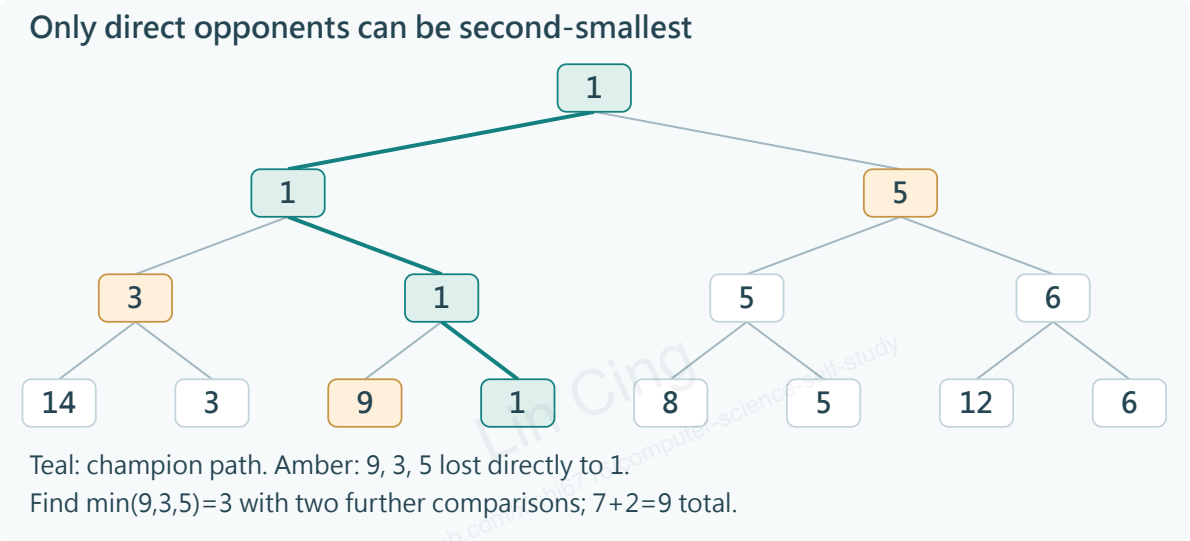
Advance the smaller key and retain each winner's directly defeated opponents. The second-smallest key must lose directly to the minimum: losing to a different key would put two keys below it. Assume distinct keys initially.

For $n \geq 2$, a balanced bracket can use byes rather than fictional keys. Eliminating one record per comparison costs $n - 1$. The champion faces $d \leq \lceil \lg n \rceil$ actual opponents; searching that list uses $d - 1$ more. The worst-case upper bound is $n + \lceil \lg n \rceil - 2$. With duplicate keys, break ties by $(key, identity)$; the result then means the second record, not necessarily the second distinct value.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Eight keys: whom did the champion defeat?




Champion $1$ directly defeated $9, 3, 5$, so the second minimum is their minimum, $3$. The tournament uses $7$ comparisons and the opponent list uses $2$, totaling $9$.

## Three-way selection removes the whole equal-key region

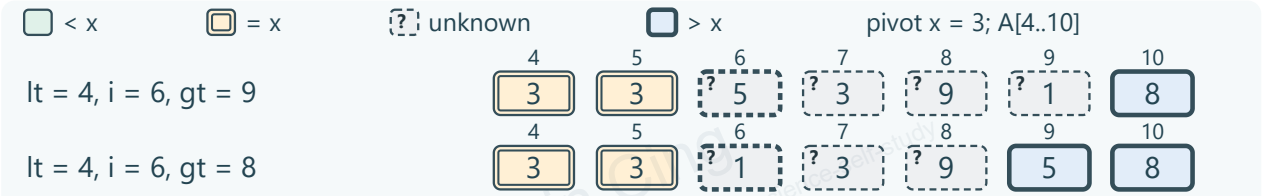
Sorting processes both sides; selection retains only the side containing the answer. Array indices use **one-based closed intervals** $A[p..r]$. Target $k$ is the local rank in this subarray, with $1 \le k \le r - p + 1$. Compare keys and exchange whole records; $x$ is a key present in the subarray.

```
PARTITION3(A,p,r,x):
    lt = p; i = p; gt = r
    while i <= gt:
        if key(A[i]) < x:
            swap(A[lt],A[i]); lt += 1; i += 1
        elif key(A[i]) > x:
            swap(A[i],A[gt]); gt -= 1
        else:
            i += 1
    return (lt,gt)
```



## Three-way partition: invariant and progress

At loop entry, $A[p..lt-1] < x$, $A[lt..i-1] = x$, $A[i..gt]$ is unknown, and $A[gt+1..r] > x$; empty ranges are permitted. A smaller key extends the less-than and equal regions; an equal key extends the equal region. A larger key swaps in an unclassified right-end record, so $i$ must stay. Each iteration decreases unknown length by one, and swaps preserve the multiset. Termination removes the unknown region.




Swap A[6] = 5 with A[9] = 1; gt becomes 8, but i stays 6.
Next classify the new A[6] = 1. Since p = lt = 4, the < x region is empty.

## Randomized selection changes the local rank only on the right

```
RANDOM-SELECT(A,p,r,k):
    require 1 <= k <= r-p+1
    while p < r:
        x = key(A[uniform-integer(p,r)])
        (lt,gt) = PARTITION3(A,p,r,x)
        L = lt-p; E = gt-lt+1
        if k <= L: r = lt-1
        elif k <= L+E: return x
        else: k -= L+E; p = gt+1
    return key(A[p])
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Selection: preserve the requested value and shrink

Because $x$ is drawn from current records, $E \geq 1$. Local ranks $L+1..L+E$ all return $x$. A left move preserves $k$; a right move subtracts the $L$ smaller and $E$ equal occurrences.

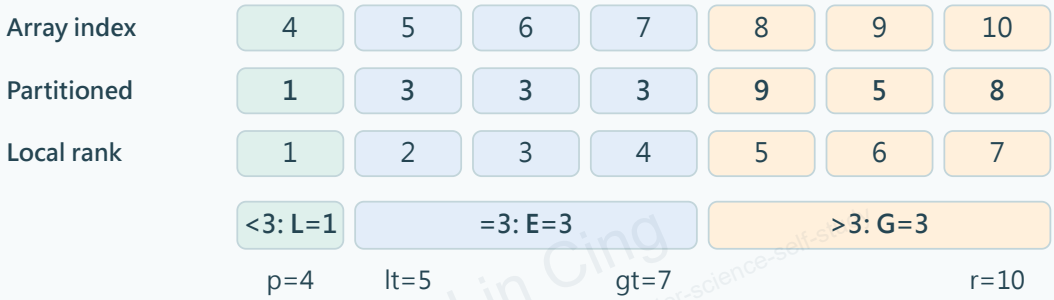
Loop invariant: the current subarray's $k$th smallest key equals the original requested key. The left branch has $1 \leq k \leq L$; the right branch has $1 \leq k - L - E \leq r - gt$, so neither retains an empty range. Every nonreturning step reduces size by at least $E$. A singleton returns its sole key, proving correctness and termination even with duplicates.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Separate local ranks, region lengths, and absolute indices

### Array indices differ from local ranks

A[4..10] = [8,3,5,3,9,1,3]; pivot x=3




k=1: left rank 1; k=2..4: return 3; k=6: right rank 6-1-3=2.

Equal indices [5,7], local ranks [2,4]; L=lt-p, E=gt-lt+1.

For $A[4..10] = [8, 3, 5, 3, 9, 1, 3]$ with $x = 3$, partition returns $lt = 5, gt = 7$, hence $L = 1, E = 3, G = 3$. Equal-region local ranks are $2..4$.



## Convert each target using region lengths

| Original local rank | Test | Next state or answer |
|---|---|---|
| 1 | $1 \le L$ | Rank 1 in $A[4..4]$ |
| 4 | $L < 4 \le L + E$ | Return 3 |
| 6 | $6 > L + E$ | Rank $6 - 1 - 3 = 2$ in $A[8..10]$ |

Comparing $k$ directly with $lt$ mixes origins: $k$ starts at the current subarray, whereas $lt$ starts at the full array's index origin. An alternative absolute target $t = p + k - 1$ stays unchanged through iterations; do not also decrement a local rank under that convention.

## Randomized selection: strong induction over all states

Conditioned on the current array, choose the pivot uniformly among its $n$ records. Future pivots are freshly uniform within their then-current subarrays. For distinct keys, local pivot rank $R$ is uniform on $1..n$. Whatever the target rank, the next size is at most

$$m_R = \max(R-1, n-R).$$

Let $F(A,k)$ be the expected remaining cost from that state. Bound current scanning and overhead by $cn$. Assume $F(B,j) \le a|B|$ for **every shorter state**. Conditioning on the chosen pivot and applying induction gives

$$F(A,k) \le cn + \frac{a}{n}\sum_{r=1}^{n} \max(r-1, n-r).$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Average the larger-side sizes explicitly

For $n = 2h$, larger-side sizes $h, \dots, 2h-1$ each occur twice: the sum is $2\sum_{t=h}^{2h-1} t = 3h^2 - h$. For $n = 2h+1$, the middle rank contributes $h$ and all other sizes occur in symmetric pairs: the sum is $h + 2\sum_{t=h+1}^{2h} t = 3h^2 + 2h$. Dividing by $n$ gives at most $3n/4$, so

$$F(A,k) \le (c + 3a/4)n \le an \qquad (a \ge 4c).$$

Choose $a$ large enough for the $n = 1$ base case. This proves $O(n)$; the initial partition scans $n > 1$ records, giving expected $\Theta(n)$ time.

No monotonicity of an unknown cost function or product of dependent expectations was assumed. The $3n/4$ bound concerns the expected larger side, not a deterministic upper bound on every partition.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Duplicates preserve expected linear time with three-way selection

For analysis only, break equal-key ties by a fixed identity order to assign unique ranks $R \in \{1, \ldots, n\}$. Actual partitioning still compares keys, not identities. Uniformly sampling a record makes this analytical rank uniform.

If the answer is in $< x$, its region has length at most $R - 1$; if in $> x$, at most $n - R$; an equal-region answer returns immediately. Every pivot identity therefore admits the same bound $m_R = \max(R - 1, n - R)$. The preceding strong induction over all smaller states applies, yielding expected $\Theta(n)$ time.

Worst-case execution remains $\Theta(n^2)$: with distinct keys, seek the minimum while choosing the current maximum every time. Scan sizes are $n, n - 1, \ldots, 2$, a possible positive-probability path. Expected and worst-case bounds differ. The iterative version with in-place three-way partition needs $O(1)$ extra space and changes its input.

All-equal input returns after one scan. Switching back to a two-way Lomuto partition that sends every equal key to one side loses this guarantee, even with random pivots.



## Deterministic selection: align code with the worked trace

All lists and sorted results below are **one-based**. SORT-COPY sorts a copy by key. STABLE-SPLIT-COPY scans the input into three fresh lists for keys less than, equal to, and greater than $x$, retaining their input order within each region. This trace-friendly version uses $O(n)$ extra space; it is not an in-place implementation.

Groups contain at most five records; take each lower median. With $g = \lceil n/5 \rceil$ medians, recursively select rank $\lceil g/2 \rceil$ as the pivot.

```
MEDIAN-PIVOT(A):
    M = empty list
    for each consecutive group G of at most 5 records in A:
        B = SORT-COPY(G)
        append key(B[ceil(length(B)/2)]) to M
    return DETERMINISTIC-SELECT(M,ceil(length(M)/2))
```

## Deterministic selection: return or recurse on one region

```
DETERMINISTIC-SELECT(A,k):
    n = length(A); require 1 <= k <= n
    if n <= 5: return key(SORT-COPY(A)[k])
    x = MEDIAN-PIVOT(A)
    (L,E,G) = STABLE-SPLIT-COPY(A,x)
    if k <= length(L):
        return DETERMINISTIC-SELECT(L,k)
    elif k <= length(L)+length(E): return x
    else:
        return DETERMINISTIC-SELECT(G,k-length(L)-length(E))
```

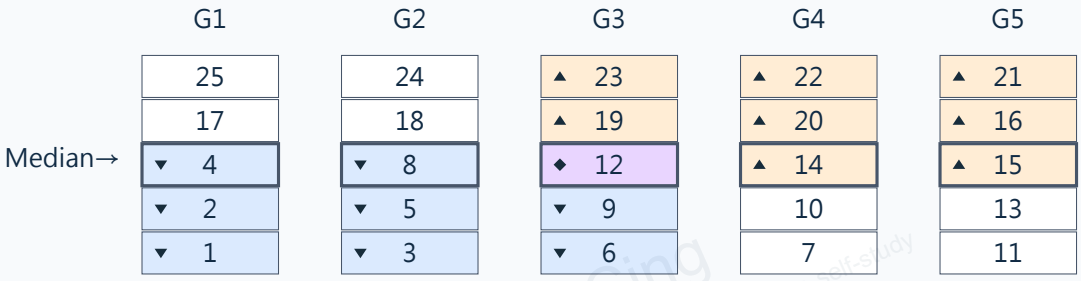
Correctness follows from region order and induction on size. Pivot recursion has size $g < n$; answer recursion has size at most $n - 1$, since $x$ is present and the equal region is nonempty. Both terminate, and both must be charged in the time analysis.



## Groups of five certify exclusions, not an exact median

### Median-of-medians: three witnesses per full group

One group per column, larger above. Middle row: group medians. Pivot x=12.




Medians [4,8,12,14,15] → third smallest 12. The overall median is 13.

▼ + ◆: 9 values ≤12 ▲ + ◆: 9 values ≥12 ◆: shared pivot

These $25$ records contain exactly keys $1..25$. Group medians are $[4, 8, 12, 14, 15]$, giving pivot $x = 12$, while the true lower median is rank $13$, namely $13$.

## List the actual witnesses on both sides

For the three groups whose medians are $\leq 12$, their smallest triples are $[1, 2, 4]$, $[3, 5, 8]$, and $[6, 9, 12]$: nine certified values $\leq 12$. For the three groups with medians $\geq 12$, their largest triples are $[12, 19, 23]$, $[14, 20, 22]$, and $[15, 16, 21]$: nine values $\geq 12$.

Overlap at $12$ is harmless. A strict greater-than recursion excludes all $\leq x$; a strict less-than recursion excludes all $\geq x$. These are alternative branches, not simultaneous claims about disjoint witnesses. With duplicates, the non-strict certificates still hold and three-way partitioning removes the entire equal region.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Rounding and incomplete groups: finish the linear-time induction

Among $g = \lceil n/5 \rceil$ group medians, their lower median $x$ has at least $\lceil g/2 \rceil$ medians on each non-strict side. Conservatively discard the possible incomplete group and the group containing the selected pivot identity. At least $\lceil g/2 \rceil - 2$ complete groups remain on either side.

Each supplies three witnesses, so each non-strict side has at least

$$3(\lceil g/2 \rceil - 2) \geq 3n/10 - 6$$

records. A strict recursive side therefore has size at most $7n/10 + 6$. A negative small-input lower bound is merely uninformative; the actual nonempty equal region still ensures strict shrinkage.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Pay for both recursive calls in the induction

Let nonrecursive work be at most $cn$. Inductively bound every shorter state by $a$ times its length. For $n \geq 140$,

$$\begin{aligned} T(n) &\leq a\lceil n/5 \rceil + a(7n/10 + 6) + cn \\ &\leq (9a/10 + c)n + 7a \leq an. \end{aligned}$$

Choose $a \geq 20c$, so $a/10 - c \geq a/20$ and $n \geq 140$ gives $an/20 \geq 7a$. Increase $a$ to cover all finite sizes $1 \leq n < 140$; this proof threshold does not change the code's size-five base case. Initial grouping costs $\Omega(n)$, establishing worst-case $\Theta(n)$ time.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Rank sixteen: trace groups, region sizes, and the next rank

Concatenate these five groups as the input, without first sorting the whole collection:

$[1, 2, 4, 17, 25]$, $[3, 5, 8, 18, 24]$, $[6, 9, 12, 19, 23]$, $[7, 10, 14, 20, 22]$, $[11, 13, 15, 16, 21]$.

| Current $(n, k)$ | Group medians $M$ and pivot $x$ | $(\lvert L\rvert, \lvert E\rvert, \lvert G\rvert)$ | Next step |
|---|---|---|---|
| $(25, 16)$ | $[4, 8, 12, 14, 15] \to 12$ | $(11, 1, 13)$ | Greater side, rank $4$ |
| $(13, 4)$ | $[19, 20, 16] \to 19$ | $(6, 1, 6)$ | Less side, rank $4$ |
| $(6, 4)$ | $[15, 16] \to 15$ | $(2, 1, 3)$ | Greater side, rank $1$ |
| $(3, 1)$ | Base sort $[16, 17, 18]$ | No partition | Return $16$ |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Reconstruct the next groups from the retained list

The first greater side retains scan order: $[17, 25, 18, 24, 19, 23, 14, 20, 22, 13, 15, 16, 21]$. Groups of five, five, and three have sorted copies $[17, 18, 19, 24, 25]$, $[13, 14, 20, 22, 23]$, and $[15, 16, 21]$. Their medians are $19, 20, 16$; a size-three base call selects $19$.

The next less side is $[17, 18, 14, 13, 15, 16]$. Its first five have median $15$; the singleton contributes $16$, so their lower median is $15$. The greater side $[17, 18, 16]$ returns rank one, $16$. This trace follows the stable-list code. In-place swaps may change region order and subsequent groups, so they need not produce the same intermediate path.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Changing group size changes the proof obligations

For fixed odd group size $s = 2t + 1$, each full group median certifies $t + 1$ records on a non-strict side. Ignoring rounding and additive constants, the two recursive-size fractions sum to

$$\frac{1}{s} + 1 - \frac{t+1}{2s} = 1 + \frac{1-t}{2s}.$$

Groups of five give $9/10 < 1$, and seven give $6/7 < 1$, leaving room to pay linear work. Incomplete groups and finite base cases still require justification. Groups of three give sum one and only the coarse bound

$$T(n) \le T(\lceil n/3 \rceil) + T(2n/3 + O(1)) + O(n).$$

This proves $O(n \lg n)$, not an $\Omega(n \lg n)$ lower bound for the actual algorithm. A lower bound needs additional evidence that such bad splits persist across levels.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Account for copied lists and the active call chain

The stable-list code uses $O(n)$ extra space: each call retains $O(n)$ copied records and executes its two recursive calls sequentially. For sufficiently large sizes, every active call chain shrinks by a fixed ratio below one, giving a geometric $O(n)$ sum; the finite small-size tail is constant. Worst-case call depth is $O(\lg n)$. In-place partitioning alone does not establish $O(1)$ total space: medians, copies, and the recursive stack also count.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Birthday collisions: build the no-collision complement

Model $k \geq 0$ people as independent uniform choices among $n \geq 1$ dates. This simplified model is not an assertion about real birthday distributions.

Conditioned on $j$ distinct dates so far, the next person has $n - j$ noncolliding choices, with conditional probability $(n - j)/n$. The chain rule gives, for $k \leq n$,

$$\Pr(\text{all dates distinct}) = \prod_{j=0}^{k-1} \left(1 - \frac{j}{n}\right),$$

$$\Pr(\text{at least one collision}) = 1 - \prod_{j=0}^{k-1} \left(1 - \frac{j}{n}\right).$$

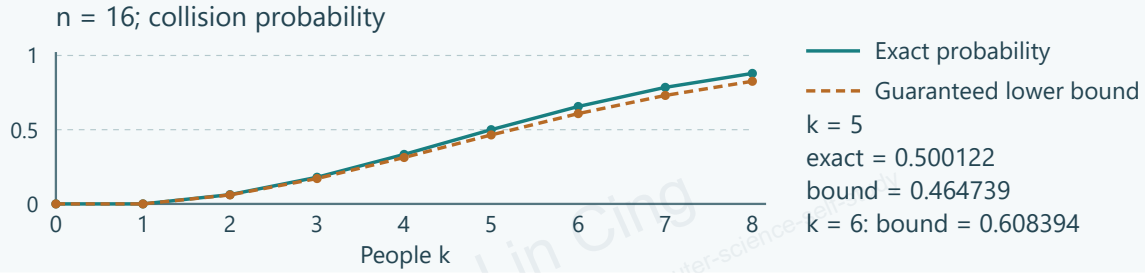
For $k = 0, 1$ there is no collision; the empty product is $1$. For $k > n$, the pigeonhole principle gives probability $1$ directly. Do not extend the product into negative factors.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A sufficient threshold can be conservative

For $2 \le k \le n$, apply $1 - x \le e^{-x}$ to each factor and take the complement:

$$\Pr(\text{collision}) \ge 1 - \exp\left(-\sum_{j=0}^{k-1} \frac{j}{n}\right) = 1 - e^{-k(k-1)/(2n)}.$$




The bound reaches $1/2$ when $k(k-1) \ge 2n \ln 2$: a **sufficient condition**, not an exact threshold. For $n = 16$, the true probability first exceeds $1/2$ at $k = 5$; the bound certifies it only at $k = 6$.



## Expected collision pairs are not collision probability

Let $I_{ij}$ indicate that people $i, j$ share a date. Independence and uniformity give

$$\Pr(I_{ij} = 1) = \sum_{d=1}^{n} \Pr(B_i = d, B_j = d) = n\frac{1}{n^2} = \frac{1}{n}.$$

The collision-pair count is $X = \sum_{i<j} I_{ij}$, hence

$$\mathbb{E}[X] = \frac{\binom{k}{2}}{n}.$$

A collision is event $X \ge 1$. Markov only yields $\Pr(X \ge 1) \le \mathbb{E}[X]$, not a reverse implication. For example, let $X = M^2$ with probability $1/M$ and zero otherwise. Its mean is $M$, but its positive-value probability is only $1/M$.

Under the birthday model, $k = o(\sqrt{n})$ makes collision probability vanish by this upper bound. For $k \ge C\sqrt{n}$ with sufficiently large fixed $C$, the exponential complement bound gives a fixed positive collision probability. Both directions establish the $\sqrt{n}$ transition scale.

## Sixteen dates: multiply every conditional probability

Take $n = 16, k = 5$ and accumulate no-collision probabilities person by person.

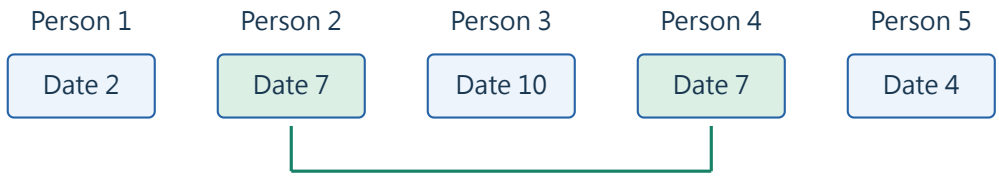
| Person added | Conditional noncollision probability | Probability all dates remain distinct |
|---|---|---|
| 1 | $1$ | $1$ |
| 2 | $15/16$ | $15/16$ |
| 3 | $14/16$ | $105/128$ |
| 4 | $13/16$ | $1365/2048$ |
| 5 | $12/16$ | $4095/8192$ |

Collision probability is $4097/8192 \approx 0.500122$, whereas expected collision pairs are $\binom{5}{2}/16 = 5/8$. At $k = 5$, the exponential complement bound $e^{-10/16} > 1/2$ cannot prove a probability above one half. At $k = 6$, $e^{-15/16} < 1/2$ can. A sufficient condition can be conservative without contradicting the exact result.



## A collision in one sample is not a model average

One date sample and its model: distinguish counts from expectations




Only colliding pair in this sample: (2,4)

Model: 16 dates, 5 people. P(collision)=4097/8192≈0.500122.

E[colliding pairs]=10/16=5/8, distinct from the event probability above.

The illustrated sample is $[2, 7, 10, 7, 4]$. Only people $2, 4$ collide, so $X = 1$ and $I_{\{X \geq 1\}} = 1$. These are values in one outcome, unlike averages $5/8$ and $4097/8192$ over all outcomes.

## Pairwise independence does not justify a joint product

Let $U, V$ be independent uniform elements of $\{0, 1, 2\}$ and define

$$B_1 = U, \qquad B_2 = V, \qquad B_3 = (2U - V) \bmod 3.$$

Every pair of dates uniquely determines $(U, V)$. The first pair is immediate; $B_1, B_3$ give $V = 2B_1 - B_3$, and $B_2, B_3$ give $U = 2(B_2 + B_3)$, modulo $3$. Hence every date pair is uniform over nine ordered pairs and is independent.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Enumerate all nine equally likely outcomes

| $U$ | $V$ | $(B_1, B_2, B_3)$ | Collision pairs $X$ |
|---|---|---|---|
| 0 | 0 | $(0, 0, 0)$ | 3 |
| 0 | 1 | $(0, 1, 2)$ | 0 |
| 0 | 2 | $(0, 2, 1)$ | 0 |
| 1 | 0 | $(1, 0, 2)$ | 0 |
| 1 | 1 | $(1, 1, 1)$ | 3 |
| 1 | 2 | $(1, 2, 0)$ | 0 |
| 2 | 0 | $(2, 0, 1)$ | 0 |
| 2 | 1 | $(2, 1, 0)$ | 0 |
| 2 | 2 | $(2, 2, 2)$ | 3 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Equal means can hide different collision probabilities

| Model | $\Pr(X=0)$ | $\Pr(X=1)$ | $\Pr(X=3)$ | $\Pr(X>0)$ | $\mathbb{E}[X]$ |
|---|---|---|---|---|---|
| Pairwise independent construction | $2/3$ | $0$ | $1/3$ | $1/3$ | $1$ |
| Three fully independent dates | $2/9$ | $2/3$ | $1/9$ | $7/9$ | $1$ |

With full independence, the $27$ equally likely triples include $6$ with all dates distinct, $3$ with all dates equal, and $3 \cdot 3 \cdot 2 = 18$ with exactly one equal pair: choose the pair, its date, then the different date. Exactly two equal pairs are impossible: equality is transitive.

Here collision probability is $3/9 = 1/3$. Three fully independent dates give $1 - 6/27 = 7/9$, yet both models have expected collision-pair count $1$. Equal expectations do not determine distributions; the exact complement product needs stronger joint assumptions.



## Balls into bins: fixed toss count, random loads

Toss $t \geq 0$ balls independently and uniformly into $b \geq 1$ bins. A fixed bin's load $N$ is the sum of $t$ indicators, so $\mathbb{E}[N] = t/b$. Choosing which $r$ tosses hit that bin gives

$$\Pr(N = r) = \binom{t}{r}(1/b)^r(1-1/b)^{t-r}, \quad 0 \leq r \leq t.$$

Let $Z_0, Z_1$ count empty bins and singleton bins. Applying indicators to each bin gives

$$\mathbb{E}[Z_0] = b(1-1/b)^t,$$

$$\mathbb{E}[Z_1] = b\left[t\frac{1}{b}(1-1/b)^{t-1}\right] = t(1-1/b)^{t-1}, \quad t \geq 1.$$

For $t = 0$, directly take $Z_0 = b, Z_1 = 0$. For $b = 1$, evaluate the events directly or interpret boundary formulas with empty-product convention $0^0 = 1$.

For $t = b = 4$, a fixed bin has mean load $1$, yet $\mathbb{E}[Z_0] = 81/64$ and $\mathbb{E}[Z_1] = 27/16$. Loads sum to four and are dependent. Mean one does not mean exactly one ball in every bin.

## Collecting every bin: wait for one new bin at each stage

Tosses are independent and uniform. For a fixed bin, let $T$ count tosses through its first hit, including the successful toss. Since $\Pr(T > t) = (1 - 1/b)^t$ for $t \geq 0$, tail summation gives

$$\mathbb{E}[T] = \sum_{t=0}^{\infty} (1 - 1/b)^t = b.$$

Four bins; wait for the first hit on bin 3

| 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|
| 2 | 2 | 4 | 1 | 4 | 3 |
| miss | miss | miss | miss | miss | hit |

T = 6

E[T] = 4

P(T = 6) = $(3/4)^5 \times (1/4) = 243/4096$

Five misses, then one hit; a single waiting time need not equal its mean.

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## Add the waiting times for successive new bins

For complete collection, after $i$ distinct bins have appeared, new-bin probability is $(b - i)/b$. Conditioned on the currently seen set, the waiting count $T_i$ has expectation $b/(b - i)$. Future tosses remain independent and uniform; elapsed time does not change this value.

Total stopping time is $C = \sum_{i=0}^{b-1} T_i$, so linearity gives

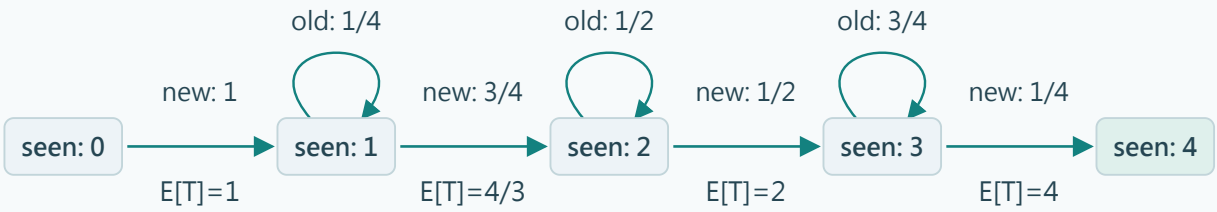
$$\mathbb{E}[C] = \sum_{i=0}^{b-1} \frac{b}{b - i} = bH_b.$$

This is $\Theta(b \ln b)$ for $b \geq 2$; for $b = 1$ it is exactly one toss. Adding expectations does not require a separate proof of independence between stages.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# Four bins: repeated hits stay in the same state

## A new bin advances; an old bin repeats the state




Tosses 2,2,4,1,4,3: new bins at tosses 1,3,4,6; actual stage lengths 1,2,1,2.

Stage means sum to 1+4/3+2+4=25/3; a particular run may stop at toss 6.

Four bins have stage means $1, 4/3, 2, 4$, summing to $25/3$. The illustrated tosses are $2, 2, 4, 1, 4, 3$, discovering new bins at tosses $1, 3, 4, 6$. Actual stage lengths $1, 2, 1, 2$ sum to six. A run's stopping time need not equal its expectation.



# First collision versus complete collection: different stopping times

First bound complete-collection time $C$. After $t$ tosses, a designated bin is empty with probability $(1 - 1/b)^t \leq e^{-t/b}$. Incomplete collection means at least one empty bin, so

$$\Pr(C > t) \leq \min\{1, be^{-t/b}\}.$$

For integer $t \geq b(\ln b + c)$ and $c \geq 0$, failure probability is at most $e^{-c}$. Empty-bin events may be dependent; the union bound is sufficient.

## Same tosses; the stopping rule changes the answer

| t = 1 | t = 2 | t = 3 | t = 4 | t = 5 | t = 6 |
|---|---|---|---|---|---|
| 2 | 2 | 4 | 1 | 4 | 3 |
| {2} | {2} | {2,4} | {1,2,4} | {1,2,4} | {1,2,3,4} |

D = 2: first repeat

C = 6: all four bins seen

Sets show distinct bins seen so far; compare D with C, not two different samples.

## First-collision time has square-root expectation

Let $D$ be the toss index of the first repeated bin. Event $D > k$ means the first $k$ tosses are distinct. For $k > b$ its probability is zero; otherwise use the birthday complement. Since $k(k-1) \geq k^2/2$ for $k \geq 2$,

$$\begin{aligned}\mathbb{E}[D] &= \sum_{k \geq 0} \Pr(D > k) \\ &\leq 2 + \sum_{k \geq 2} e^{-k^2/(4b)} \\ &\leq 2 + \int_0^\infty e^{-x^2/(4b)} dx = 2 + \sqrt{\pi b}.\end{aligned}$$

For a lower bound, integer $k \leq \sqrt{b}/2$ has collision probability at most $k(k-1)/(2b) \leq 1/8$. Thus the first $\Theta(\sqrt{b})$ tail terms are at least $7/8$, giving $\mathbb{E}[D] = \Omega(\sqrt{b})$.

Consequently $\mathbb{E}[D] = \Theta(\sqrt{b})$, whereas $\mathbb{E}[C] = \Theta(b \ln b)$. The same balls-and-bins model has different answers for first repetition and complete coverage.



## Heads streaks: window count and longest length differ

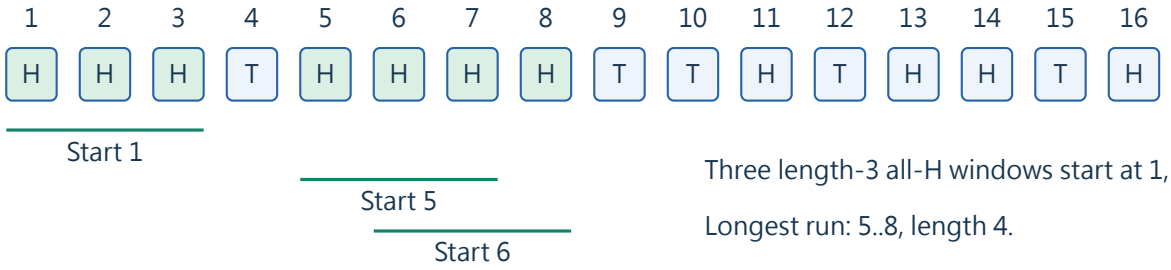
Flip a fair coin independently $n$ times. Let $L$ be the longest consecutive-heads length, with $L = 0$ when no head occurs. For $1 \leq k \leq n$, event $A_{i,k}$ means positions $i$ through $i + k - 1$ are all heads; valid starts satisfy $1 \leq i \leq n - k + 1$. Thus

$$X_k = \sum_{i=1}^{n-k+1} \mathbf{1}_{A_{i,k}}, \qquad \mathbb{E}[X_k] = (n - k + 1)2^{-k}.$$

Overlapping windows need not be independent. For example, $\Pr(A_{1,3} \cap A_{2,3}) = 2^{-4} = 1/16$ because only four heads are needed, whereas the product of marginals is $2^{-6} = 1/64$. Linearity of expectation remains valid; exact union probabilities require accounting for overlaps.

## Count windows without confusing them with runs

Four consecutive Hs contain two dependent length-3 windows




Three length-3 all-H windows start at 1,5,6.

Longest run: 5..8, length 4.

Overlapping windows can both occur; starts 5 and 6 share two flips.

The illustrated length-three windows start at $1, 5, 6$, giving $X_3 = 3$. The heads run at positions $5..8$ contributes two of these windows. Its longest length is $L = 4$, distinct from both the window count and the number of heads runs.



## Upper bounds: from streak tails to expected length

Event $\{L \geq k\}$ is the union of all all-heads windows of length $k$. For $1 \leq k \leq n$, the union bound gives

$$\Pr(L \geq k) \leq \min\{1, (n - k + 1)2^{-k}\} \leq \min\{1, n2^{-k}\}.$$

For $n \geq 1$, put $k_0 = \lceil \lg n \rceil$. Integer tail sums and a geometric series yield

$$\begin{aligned} \mathbb{E}[L] &= \sum_{k=1}^{n} \Pr(L \geq k) \\ &\leq k_0 + \sum_{k=k_0+1}^{\infty} n2^{-k} = k_0 + n2^{-k_0} \leq \lceil \lg n \rceil + 1. \end{aligned}$$

Here $\lg$ denotes base-two logarithm. For $k > n$ the event is impossible; for $n = 0$, set $L = 0$ directly without taking a logarithm.

For $r \geq 1$, choosing integer threshold $k = \lceil r \lg n \rceil$ gives $\Pr(L \geq k) \leq n^{1-r}$. At $n = 1000, k = 20$, the sharper bound is $981/2^{20} < 10^{-3}$; at $k = 30$, even $1000/2^{30} < 10^{-6}$ suffices. These are probability guarantees for an unusually long streak in one run.

## Lower bound: disjoint blocks supply independence

For sufficiently large $n$, let $s = \lfloor (\lg n)/2 \rfloor \geq 1$ and $m = \lfloor n/s \rfloor$. Partition the first $ms$ flips into $m$ blocks of length $s$, discarding the incomplete final block. Each block is all heads with probability $2^{-s} \geq n^{-1/2}$. Disjoint blocks use independent flips, so their success events are mutually independent.

If $L < s$, every block must fail. The converse need not hold: a long run can cross a block boundary. Consequently,

$$\Pr(L < s) \leq (1 - 2^{-s})^m \leq e^{-m2^{-s}} \leq e^{-m/\sqrt{n}} \longrightarrow 0.$$

Since $m = \Theta(n/\lg n)$, we have $m/\sqrt{n} \to \infty$. Nonnegativity then gives

$$\mathbb{E}[L] \geq s \Pr(L \geq s) = \Omega(\lg n).$$

Combining with the upper bound proves $\mathbb{E}[L] = \Theta(\lg n)$. Many overlapping windows in expectation alone would not justify the high-probability existence step; independence of the selected blocks does.

For $n = 16, s = 2, m = 8$, at least one designated block is HH with probability $1 - (3/4)^8 = 58975/65536$. Hence $\mathbb{E}[L] \geq 117950/65536$, a checkable lower bound rather than an exact mean.



## Online selection: specify information and decisions

There are $n \geq 2$ candidates with distinct abilities, arriving in a uniformly random permutation. Interviews permit comparisons only with candidates already seen; rejected candidates cannot be recalled, and at most one hire is allowed. The objective is to select the globally best candidate, rather than minimize interview cost.

Choose integer $1 \leq k < n$. Reject the first $k$ candidates while recording their highest score $M$, then accept the first later score exceeding $M$. If none does, accept the last candidate.

The second loop need not update $M$: before the first threshold crossing every observed score is at most $M$, and a crossing returns immediately. The final fallback accepts the current last candidate, without recalling anyone. Handle $n = 1$ by selecting its sole candidate; define $k = 0$ separately as selecting the first candidate.

## Online selection: code and stopping invariant

```
ONLINE-SELECT(n,k):
    require n >= 2 and 1 <= k < n
    M = -infinity
    for i = 1 to k:
        M = max(M, score(i))      // interview, then reject
    for i = k+1 to n:
        if score(i) > M: return i
    return n
```

Before inspecting each later candidate, no earlier candidate has crossed the sample maximum $M$. A crossing ends the search immediately; if none occurs, the fallback accepts the last candidate.



## Success probability: condition on the best arrival

Condition on the globally best candidate arriving at position $i$. If $i \le k$, rejection during observation makes success impossible. If $i > k$, success occurs exactly when the best of the first $i-1$ candidates lies among the first $k$: otherwise a later candidate would already have crossed $M$, while if it does, candidate $i$ is the first crossing.

The best position is uniform, with probability $1/n$. Conditional on that position, the first $i-1$ candidates still have a uniform relative order. Their maximum lies among the first $k$ with conditional probability $k/(i-1)$. Summing disjoint position events gives

$$P_k = \sum_{i=k+1}^{n} \frac{1}{n}\frac{k}{i-1} = \frac{k}{n}\sum_{j=k}^{n-1}\frac{1}{j}.$$

This uses conditional probabilities, without assuming independence of the two maximum-position events.

Define boundaries separately: for $n = 1$ success is certain; for $k = 0$ choosing the first candidate succeeds with probability $1/n$. The harmonic sum formula applies only to $k \ge 1$.

## Optimal threshold: finite differences before limits

Put $q_k = \sum_{j=k}^{n-1} 1/j$, with empty sum $q_n = 0$. Since $q_k = 1/k + q_{k+1}$, for $1 \le k \le n-2$,

$$P_{k+1} - P_k = \frac{(k+1)q_{k+1} - kq_k}{n} = \frac{q_{k+1} - 1}{n}.$$

The strictly decreasing sequence $q_k$ makes success unimodal. A threshold satisfying $q_k \ge 1 \ge q_{k+1}$ maximizes it; equality can produce adjacent tied optima. For $n = 2$, the sole positive threshold $k = 1$ ties $k = 0$ at success $1/2$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Why the optimal success rate tends to 1/e

Integral bounds for decreasing $1/x$ yield

$$\ln(n/k) \le q_k \le \ln(n/k) + 1/k.$$

Thus $f(k/n) \le P_k \le f(k/n) + 1/n$, where $f(x) = x\ln(1/x)$. Derivatives $f'(x) = \ln(1/x) - 1$ and $f''(x) = -1/x < 0$ give maximum $1/e$ at $x = 1/e$.

Choose the nearest legal integer to $n/e$, making $k/n$ differ from $1/e$ by $O(1/n)$. Continuous differentiability gives $f(k/n) = 1/e - O(1/n)$, while every threshold satisfies $P_k \le 1/e + 1/n$. Optimal success therefore converges to $1/e$. At finite $n$, use the exact difference rather than treating $n/e$ as an exact integer solution.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Six candidates: verify thresholds and individual decisions

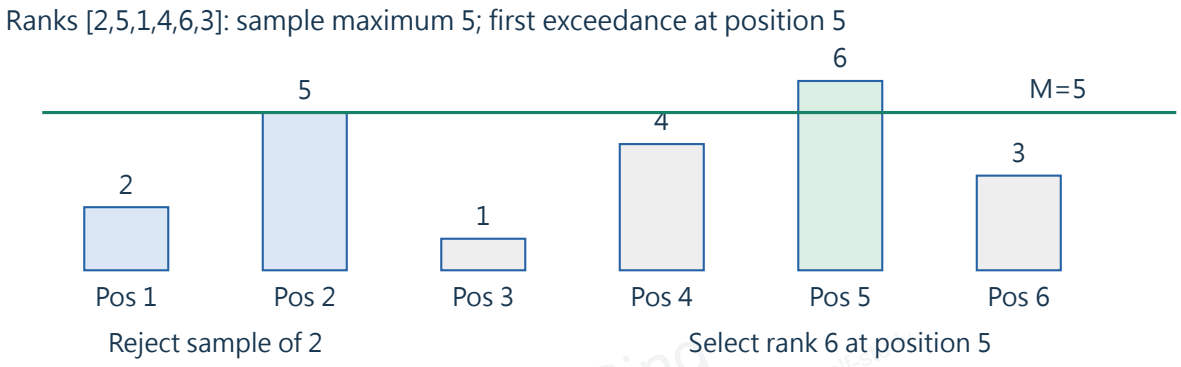
For $n = 6$, substitute each threshold into $P_k = (k/6)q_k$:

| $k$ | $q_k$ | Exact success probability $P_k$ |
|---|---|---|
| 1 | $137/60$ | $137/360$ |
| 2 | $77/60$ | $77/180$ |
| 3 | $47/60$ | $47/120$ |
| 4 | $9/20$ | $3/10$ |
| 5 | $1/5$ | $1/6$ |

Since $q_2 > 1 > q_3$, the best positive threshold is $k = 2$; threshold $0$ succeeds with probability only $1/6$.



## Follow the threshold through every candidate




For ranks $[2, 5, 1, 4, 6, 3]$, reject the first two and set $M = 5$. Reject candidate three because $1 \le 5$, and four because $4 \le 5$. Candidate five has $6 > 5$ and is accepted immediately; candidate six need not be interviewed. For failure example $[5, 6, 1, 2, 3, 4]$, the best candidate is rejected during observation, nobody later exceeds $6$, and the fallback accepts rank $4$.

## Random recursive splitting: conditioning and leaf conservation

Consider a separate cost model. A call of size $n \geq 2$ prints $n$ stars, chooses $p$ uniformly from $\{1, \ldots, n-1\}$, and processes sizes $p, n-p$. A size-one call prints one exclamation mark and stops. Every draw is uniform conditional on prior history.

Let $L_n, S_n$ denote expected exclamation marks and stars, with $L_1 = 1, S_1 = 0$. Condition on $p$, add the two expected costs, then average over $p$. Symmetry under $j \leftrightarrow n-j$ gives

$$L_n = \frac{2}{n-1} \sum_{j=1}^{n-1} L_j, \qquad S_n = n + \frac{2}{n-1} \sum_{j=1}^{n-1} S_j.$$

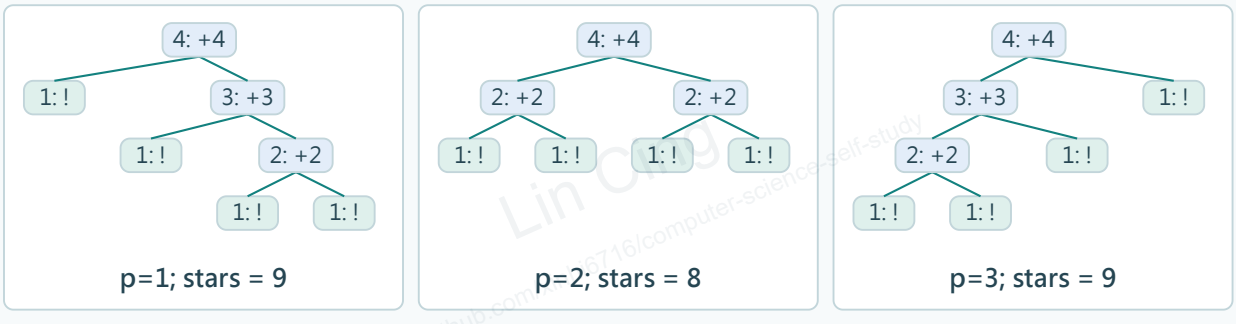
Linearity needs no independence between the two subtree costs. Every branch strictly decreases its positive integer size, ensuring termination.



## Four items: the same leaf count, different work

Splitting conserves total size and all final leaves have size one, so **every execution has exactly $n$ leaves**, not merely $n$ in expectation. Star count varies. At $n = 4$, a balanced split costs $4 + 2 + 2 = 8$. A $1 + 3$ split costs $4 + (3 + 2) = 9$. Averaging the three equally likely root splits gives $S_4 = (9 + 8 + 9)/3 = 26/3$.

Three equally likely root splits; four leaves in every tree



## Solving split cost: subtract adjacent sizes

Strong induction verifies the leaf mean: if $L_j = j$ for $j < n$, then $L_n = \frac{2}{n-1}\sum_{j<n} j = n$.

For stars, clear the denominators:

$$(n-1)S_n = n(n-1) + 2\sum_{j<n} S_j,$$

$$nS_{n+1} = n(n+1) + 2\sum_{j<n} S_j + 2S_n.$$

Subtract to obtain $nS_{n+1} - (n-1)S_n = 2n + 2S_n$, hence $S_{n+1} = 2 + \frac{n+1}{n}S_n$. The subtraction applies for $n \geq 2$; at $n = 1$, check the difference directly from $S_1 = 0, S_2 = 2$.

With $Q_n = S_n/n$, we have $Q_{n+1} = Q_n + 2/(n+1)$. Sum from $Q_1 = 0$ to get

$$Q_n = 2(H_n - 1), \qquad S_n = 2n(H_n - 1) = \Theta(n \ln n).$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



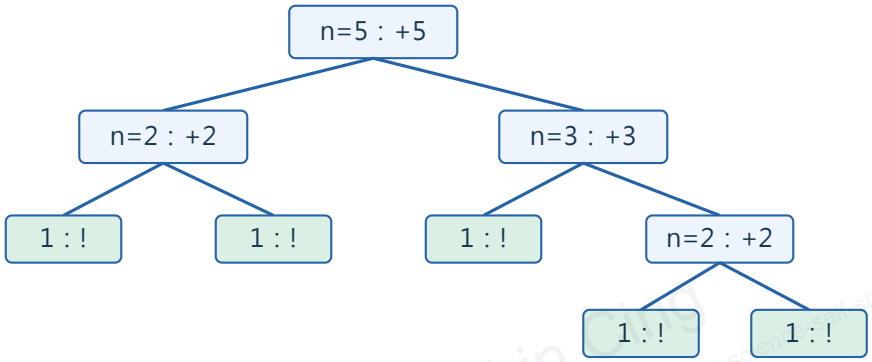
## Verify the closed form on small inputs

| $n$ | Leaves in every execution | $\mathbb{E}[\text{stars}]$ |
|---|---|---|
| 2 | 2 | 2 |
| 3 | 3 | 5 |
| 4 | 4 | $26/3$ |
| 5 | 5 | $77/6$ |
| 6 | 6 | $87/5$ |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Expected cost and worst execution are different

Five unit leaves: leaf count is conserved; star cost varies




Leaves here=5

Stars here=12

E[stars]=77/6

+number: stars at this call

!: one exclamation mark

Shown: p=2, then 3=1+2. All splits give 5 leaves; total stars depend on splits.

The worst execution splits off one each time, printing $2 + 3 + \cdots + n = n(n+1)/2 - 1$ stars. The expected linearithmic bound is not a bound on every execution.



## References and attribution

Reference textbook: Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition, Chapters 5–9. Supplementary notes: MIT 6.046J/18.401J, Fall 2005. Lecture 4: Quicksort (Charles E. Leiserson, 2005-09-21); Lecture 5: Sorting Lower Bounds / Linear-Time Sorting (Erik Demaine, 2005-09-26); Lecture 6: Order Statistics (Erik Demaine, 2005-09-28).

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study